

Advanced Data Analysis (Assessment 2)

Data Mining

Student Number: 100518675

Statement of AI usage: *AI was utilised to identify attributes, determine suitable methods for data preprocessing and model training, and inform the overall structure of the report.*

1. Executive Summary

This report looks at over 12,000 customer service interactions to answer a key question: can we tell in advance when a customer complaint is likely to escalate into something serious? The data covered four possible outcomes, no escalation (48.9%), minor escalation (30.4%), serious escalation (16.7%), and legal escalation (4.0%). Several predictive models were tested, and the best performer, Gradient Boosting, correctly identified the likely outcome 96% of the time. To understand why, a tool called SHAP was used to highlight which factors mattered most. Across the most serious cases, the clearest warning signs were: an existing risk indicator on the customer's account, service pressure score, customer engagement score, wait time, total burden, and sentiment score, with the relative importance varying depending on whether a serious or legal outcome was being predicted. When the data was grouped without prior assumptions, one natural cluster stood out, with 55% going on to have a serious escalation and 22% reaching a legal dispute. The key takeaway is that escalation is predictable and largely preventable. Customers who escalate tend to have a history of poor service experiences building up over time, meaning there is a real opportunity to step in early before situations reach a serious or legal stage.

2. Introduction

The goal of this analysis is to provide insight into whether customer escalations can be predicted before they happen, and to identify what factors drive them. This report applies supervised machine learning and unsupervised clustering to build an early warning system for escalation risk. The report walks through data exploration and visualisation, pre-processing and feature engineering, model selection and evaluation, and clustering analysis. This report is structured as follows: Advanced Data Analysis Report, Jupyter Notebook (Appendix A), Data Dictionary (Appendix B), and Travel Log (Appendix C).

3. Data Exploration, Visualisation, and Dictionary

3.1 Data Exploration

The raw dataset contains 12,362 rows and 41 columns, representing individual customer interactions. The target variable is the escalation level and has four classes including: No escalation (48.9%), minor escalation (30.4%), serious escalation (16.7%), and legal escalation (4.0%). A random sample of 10 records was inspected to get a sense of the data, confirming the presence of both categorical and numerical attributes ranging from customer demographics to operational performance metrics. The dataset initially comprises 15 object (categorical) columns, 7 integer columns, and 19 float columns. Several columns have missing values: sentiment score (5.10%), customer engagement score (6.68%), agent experience years (1.80%), subscription value str (3.23%), and risk indicator and service pressure score each have around 1.46-1.51% missing. The highest age value is 663, which is an obvious outlier error. The wait time seconds max is 1520s (approx. 25 minutes), and resolution time minutes max is 470. These may be

potential outliers. The region column has 47 unique values, suggesting there may be errors due to high cardinality or noise.

Based on the info retrieved and the column structure, the dataset's 41 features can be grouped as follows: **Target Variable:** escalation level (object): No escalation, Minor escalation, Serious escalation, and Legal escalation. **Customer Demographics:** age (float64): customer age in years; tenure months (int64): number of months the customer has been with the service; loyalty status (object): membership tier (Bronze, Silver, Gold, Platinum); segment (object): customer segment (e.g., Retail, Student, Enterprise). **Interaction Context:** region (object): geographic region of the customer; industry (object): industry sector; channel (object): contact channel (Phone, Chat, Email, In-app); issue category (object): broad category of the issue (e.g., Billing, Technical, Account, Orders, Returns); issue subtype (object): specific sub-category of the issue; product line (object): product type; device type (object): device used; time of day bucket (object): time of day when contact was made. **Operational Metrics:** wait time seconds (int64): time spent waiting before agent connection; resolution time minutes (int64): time taken to resolve the issue; previous interactions 30d (int64): number of contacts in the past 30 days; agent experience years (float64): years of experience of the handling agent; first contact resolution (object): whether the issue was resolved on first contact (Yes/No); number of transfers (int64): number of times the call or chat was transferred; SLA breaches last 6m (int64): number of SLA breaches in the past six months; lifetime tickets (int64): total historical tickets raised by the customer; average response delay (float64): average delay in agent responses. **Customer Value and Risk:** sentiment score (float64): sentiment of the interaction; monthly fee (float64): customer's monthly subscription fee; subscription value str (object): string representation of subscription value; churn risk band (object): categorical churn risk (e.g., Low, Medium, High); priority flag (object): whether the customer is flagged as high priority (Yes/No). **Behavioural and Telemetry Features:** behaviour score 1 through behaviour score 5 (float64): five composite behavioural indicators scored 0-100; ops telemetry kpi 1 through ops telemetry kpi 5 (float64): five standardised operational KPI metrics. **Derived Scores:** post interaction quality score (float64): quality score assigned after interaction; risk indicator (float64): a computed escalation risk signal; service pressure score (float64): a measure of service load or pressure; customer engagement score (float64): a composite engagement metric.

Initial inferences at this stage include customer engagement score being identified as the most powerful individual predictor, and operational service metrics such as wait time, number of transfers, monthly fee, agent experience, and SLA breaches last 6m being weaker individual indicators. This is notable as it challenges that these poor service operations directly drive escalation. However, these should not be ruled out completely. The exploration reveals an imbalanced dataset, and targeted data quality work on missing values and extreme outliers is needed before modelling begins.

3.2 Visualisations

The escalation level distribution (**Appendix A, Figure 1**) count plot confirms the class imbalance visually. The bar for “No escalation” is notably larger than “Minor escalation”, which overshadows “Serious escalation” and “Legal escalation”. The missingness heatmap (**Appendix A, Figure 2**) shows missingness correlation across columns, which variables tend to be missing together. The critical finding is a 0.9 missingness correlation between sentiment score and customer engagement score. A correlation this high means these two fields are almost always missing together for the same set of records. The loyalty status also shows a slight correlation (0.1) with age missingness. These two columns are missing largely independently of each other. The boxplots showing some numeric features by escalation (**Appendix A, Figure 3**) compare wait time seconds, number of transfers, monthly fee, agent experience years, and SLA breaches last 6m across escalation levels. The inference here is that these metrics are not strong for individual predictors of escalation severity because it shows nearly identical distributions across escalation levels.

Next are the boxplots showing sentiment and engagement by escalation level (**Appendix A, Figure 4**). For sentiment score, a flat median is displayed across all classes (around 0). Surprisingly, customer sentiment seems not to cleanly separate escalation levels. This may reflect that sentiment is captured after the issue is partially resolved. For customer engagement score, there is a clear increasing trend with escalation severity. Customers with No escalation have a median around 4, Minor escalation around 7, Serious escalation around 10, and Legal escalation around 13. This is a clear numeric signal of escalation risk found in the exploration and suggests this feature will be highly predictive. In other words, for customer engagement score, the feature distribution for each class is distinct from one another. For sentiment score, the classes do not separate cleanly. For example, if a customer’s sentiment score is -0.3, you cannot deduce which escalation class they belong to because all four classes have their median at roughly 0. The distributions are overlapping almost completely. Whereas, for engagement, a score of 13 means it is common in legal escalation cases and rare in no escalation cases.

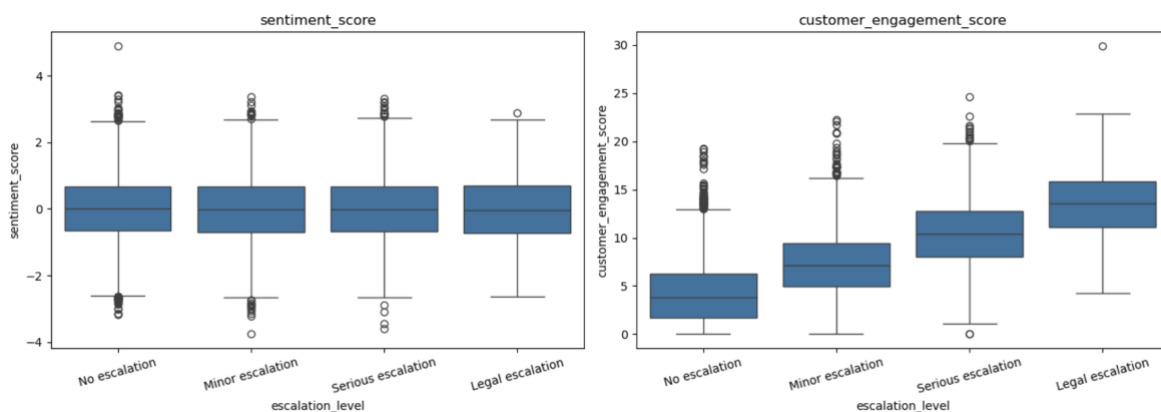


FIGURE 4: SENTIMENT AND ENGAGEMENT VS ESCALATION LEVEL

The histogram displaying behaviour scores (**Appendix A, Figure 5**) shows approximately a normal bell-shaped distribution centered around 40-55 with a range of 0-100. They appear to have a similar variance (approx. 15 std). Since this seems to appear identically distributed, a check to see whether they are independent or correlated before using as features would be appropriate.

3.3 Data Dictionary

A data dictionary table was created to summarize each column's data type, not-null count, null count, null percentage, and unique value count. This table serves as a structured reference throughout the analysis. Other attributes are appended after data cleaning and transformation to show new and accurate data.

4. Data Cleaning, Feature Engineering and Pre-Processing

4.1 Data Cleaning and Pre-Processing

For data types, subscription value str stores monetary values as strings, which cannot be used in modelling. To fix it, `.str.replace()` strips the currency symbol and commas using regex, then `.astype(float)` converts the cleaned string to a number. The column is then renamed to subscription value. Next, the first contact resolution column is stored as a generic object type. It is converted to a binary integer by mapping "Yes" to 1 and "No" to 0, with the resulting counts showing 8,922 positive resolutions and 3,440 negative ones. This will allow calculating a resolution rate with mean, use it in a model, or aggregate it easily. Finally, the priority flag column is verified whether it is binary (Yes/No), revealing 10,067 "No" entries and 2,295 "Yes" entries, confirming that it could similarly be encoded as a binary variable for further analysis.

Two columns need string cleaning: region and issue subtype. Both are object columns with inconsistent spellings, typos, and near-duplicates. The "thefuzz" library fuzzy matches each value against a list of valid entries and replaces it with the closest match. Regions (London, South East, Wales, Scotland, North West, Midlands) are stored as valid regions; and issue subtypes are stored in valid issue subtypes with the cleaned entries. After correcting the two columns, a check for duplicates is applied, and duplicates are dropped to eliminate redundancy.

Numeric columns were checked for outliers using a function with the IQR method. Rather than removing rows, extreme values were capped at the IQR bounds ($Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$) to preserve records while limiting the influence of extremes on model training. This was applied to the majority of numeric columns, including operational metrics such as wait time seconds, number of transfers, and SLA breaches last 6m. Three columns were left uncapped including tenure months, agent experience years, and age (after the >120 correction), as outliers in these represent plausible values.

Several columns had missing values. For categorical columns (loyalty status, time of day bucket), for which mean and median are not applicable, had missingness below 0.2% and were

imputed using the mode, the most frequently occurring value. Numeric columns (age, agent experience years, risk indicator, service pressure score, subscription value) had missingness between 0.25% and 1.8% and were imputed using the median, which is less sensitive to extreme values than the mean. Sentiment score and customer engagement score required separate handling. A missingness correlation of 0.9 indicates they go missing together, likely from the same data source. A binary flag was created before imputation to preserve this signal. A record with no sentiment data may reflect an abruptly ended interaction or system failure, both of which could correlate with escalation outcome. It is confirmed that 11,422 rows had complete data for both sentiment score and customer engagement score; 818 rows had at least one of those values missing and were flagged with a 1. Those 818 records are now imputed, but the flag preserves the fact that their sentiment data was originally absent.

To address leakage risks, post interaction quality score, resolution time minutes, and first contact resolution were dropped, as all three are known once the interaction has concluded and would not be available at prediction time. Checking the target variable distribution confirmed class imbalance. Once more, nearly half of the dataset (48.9%) belongs to the no escalation class, while legal escalation accounts for only 4% of the data.

4.2. Feature Engineering

Three features were engineered from existing variables. The total burden variable adds number of transfers and SLA breaches into a single column. Both measure operational failure. For example, a customer being passed around and deadlines being missed. Combining them produces one score representing how badly an interaction was handled. The high-risk variable is a binary flag set to 1 only when a customer is flagged as high churn risk and their interaction is marked as priority, identifying customers who are already at risk of leaving and have been internally flagged as important. The recent interaction ratio variable divides contacts in the last 30 days by lifetime tickets, capturing whether a customer's contact frequency has increased relative to their history. For example, a customer who has made 10 contacts in the last 30 days out of 11 lifetime total is going to behave differently than one with 10 contacts out of 500.

Since models require numeric input, categorical columns were encoded before training. The target variable, escalation level, was encoded into integers 0 through 3 using LabelEncoder to convert the four classes into numeric form. Priority flag was already confirmed as binary integer during the data type fixing step. The remaining nominal columns including channel, issue category, issue subtype, product line, device type, region, industry, segment, time of day bucket, churn risk band, and loyalty status were encoded using `pd.get_dummies`, which creates a separate binary column for each category. Redundancy is avoided by removing one category per column. When all remaining dummy columns are 0, the models infer to the dropped category by elimination.

4.3. Train/Test Split and Scale (Oh My!)

This is where the train/test split happens on the cleaned dataset. The total cleaned dataset has 12,240 rows since duplicates and problematic rows were removed during cleaning. After importing the necessary sklearn libraries, X is defined as all feature columns and y as the encoded escalation level (escalation level enc). The split produces 76 features (columns) in X. The resulting split produces 9,792 rows for training and 2,448 rows for testing, meaning 80% of the data is used for training. The stratify = y is applied because the dataset is imbalanced, with the legal escalation class making up only 4% of the data. Without stratification, this class could be underrepresented, or missing entirely, from the test set. Setting random state = 42 ensures the split is reproducible across runs. For scaling, the scaler is fit only on X train, learning the mean and range from the training data, and the same transformation is then applied to X test. Although not all models require scaling, it is applied here as a precaution in case it is needed later in the modelling process.

5. Supervised Model Training, Tuning and Evaluation

Four models were built to predict how serious a customer's complaint would become: no escalation, minor, serious, or legal. The main challenge was that most cases were "no escalation", while legal escalation made up only 4% of the data. To stop models from just ignoring rare cases, all models were set up to treat every category fairly. Three of the four models were also tuned by testing many different settings automatically and keeping the best ones.

- **Logistic Regression** was the starting point. Despite its simplicity, it performed well, with an overall accuracy of 96% on the test set (**Appendix A, Figure 6**).
- **Decision Tree** was the weakest model, scoring 92.6% accuracy. The problem was that it memorised the training data too well, when given new data it hadn't seen before, its performance dropped. It was best at catching legal escalation cases but worse overall. Its main usefulness is that it's easy to explain (**Appendix A, Figures 7 and 8**).
- **Random Forest** scored 94.7% accuracy, sitting in the middle. Performed well at avoiding the memorisation problem. Interestingly, it still couldn't beat the Logistic Regression, which suggests the data itself is relatively straightforward and doesn't need a complicated model. Its feature importance chart was the most useful output; it clearly showed that the risk indicator is the most important predictor, followed by service pressure score and customer engagement score (**Appendix A, Figures 9 and 10**).

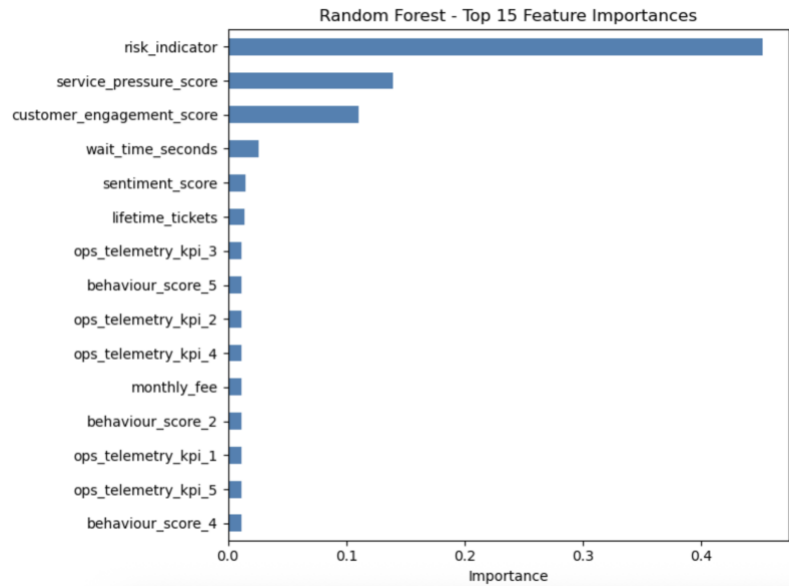


FIGURE 10: RANDOM FOREST - TOP 15 FEATURE IMPORTANCES

- **Gradient Boosting** was the best model, scoring 96.3% accuracy. This made it good at catching legal escalations, improving that score from 0.88 to 0.95, the biggest improvement across all models. The matrix shows exactly where the best model makes mistakes (91 correct, 5 misclassified as serious) (**Appendix A, Figure 11**).

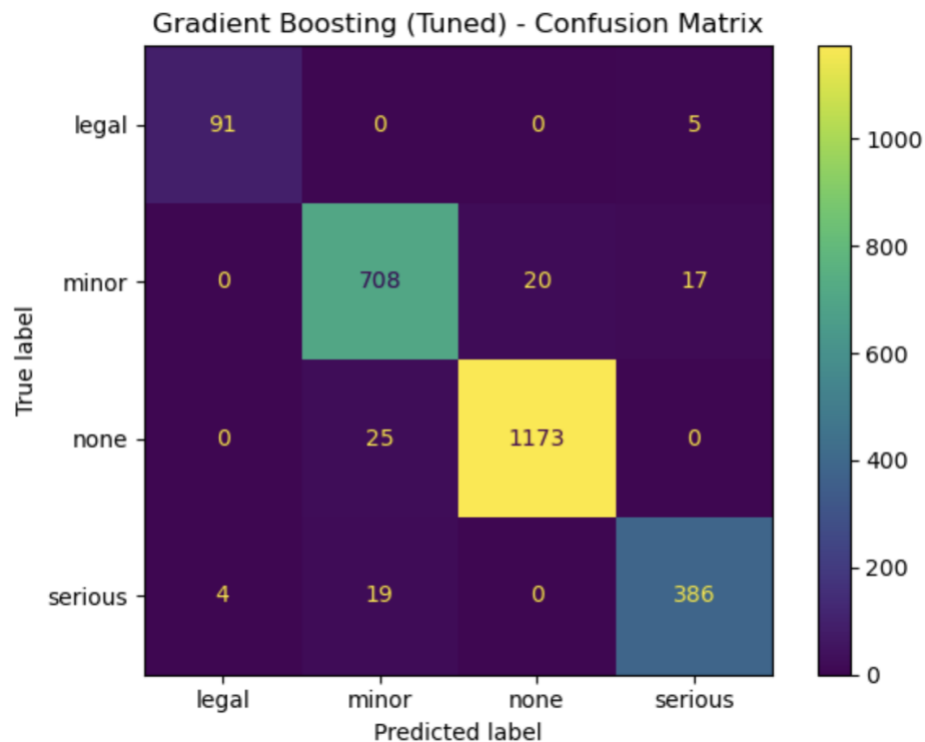


FIGURE 11: GRADIENT BOOSTING CONFUSION MATRIX

The **SHAP analysis** was used to look inside the Gradient Boosting model and understand why it makes certain predictions. The finding was clear and consistent across all four escalation levels: risk indicator and service pressure score are the two features that matter most. High values of both almost always mean escalation is coming. Below those two, wait time, customer engagement, sentiment score, and lifetime tickets also play a role (**Appendix A, Figure 13**).

Overall, every model scored over 92% accuracy, which shows the data contains enough information to predict escalation well. A business could realistically use risk indicator and service pressure score as early warning signals to flag customers before a situation escalates. The model comparison chart showcases each model's performance (**Appendix A, Figure 12**).

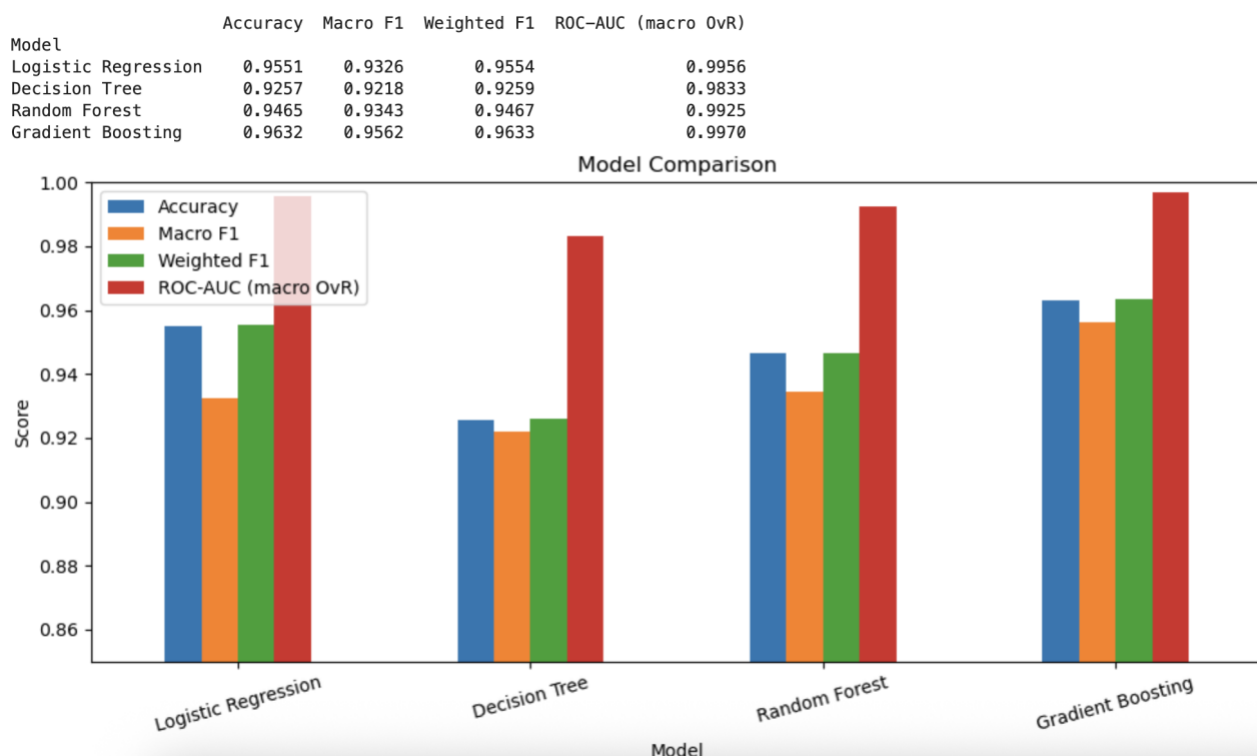


FIGURE 12: MODEL COMPARISON CHART

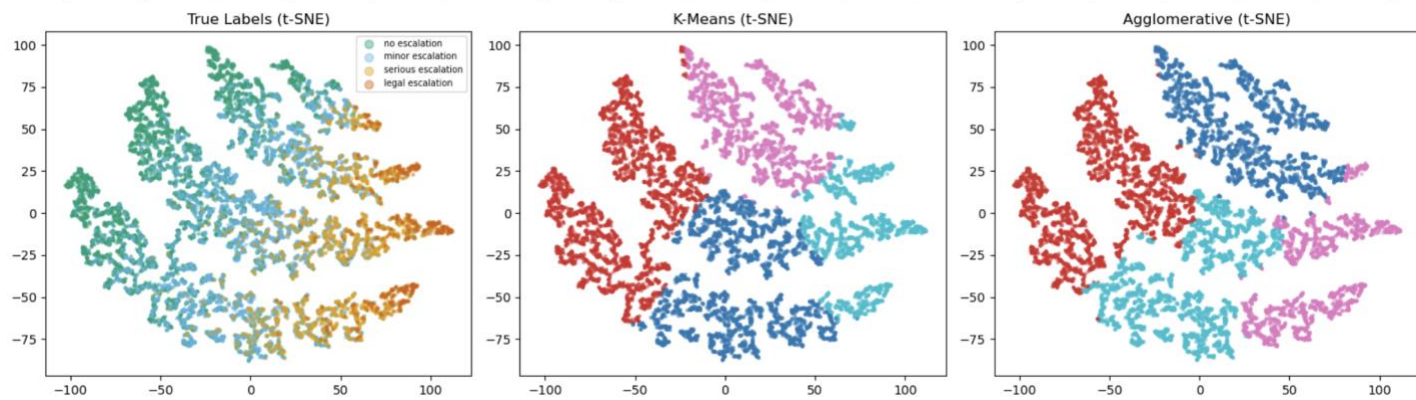
6. Unsupervised Learning Using Clustering Algorithms

The goal is to discover natural customer groupings without using escalation labels and then see whether those clusters align with escalation behavior, using Silhouette to measure separation and ARI to measure label alignment. As shown in [Table 1](#), both K-Means and Agglomerative produce low to moderate scores, while DBSCAN collapses into a single cluster, indicating weak structure and limited alignment with true escalation outcomes. This is expected because escalation risk exists on a spectrum, and the data is broadly distributed rather than clustering into clear, dense groups.

Table 1

Method	Silhouette	ARI vs True Labels
K-Means	0.35	0.21
Agglomerative	0.30	0.18
DBSCAN	N/A	(1 cluster found)

The cluster structure illustrates the comparison between the true escalation labels and the algorithm outputs. Because PCA compresses the data, the resulting scatter plot appears as a cloud, which is expected when groups are not cleanly separable. In contrast, t-SNE better shows relationships and reveals more distinct groupings in the lower row (**Appendix A, Figure 14**).

**FIGURE 14: VISUALISING CLUSTER STRUCTURE (LOWER ROW)**

The cluster to escalation is key because it directly addresses how the discovered clusters relate to escalation levels by showing their composition (**Appendix A, Figure 15**). The heatmap displays the percentage of each escalation level within each K-Means cluster. Notably, cluster 1 contains 84% no escalation cases, clearly identifying a low-risk group, while cluster 3 stands out with 55% serious and 22% legal escalations. Evidence that the model identified a high-risk customer group without seeing the labels. This indicates that the underlying features carry meaningful signals about escalation risk. The K-Means is captured more clearly than the Agglomerative clustering, which produces more blended and less interpretable groups.

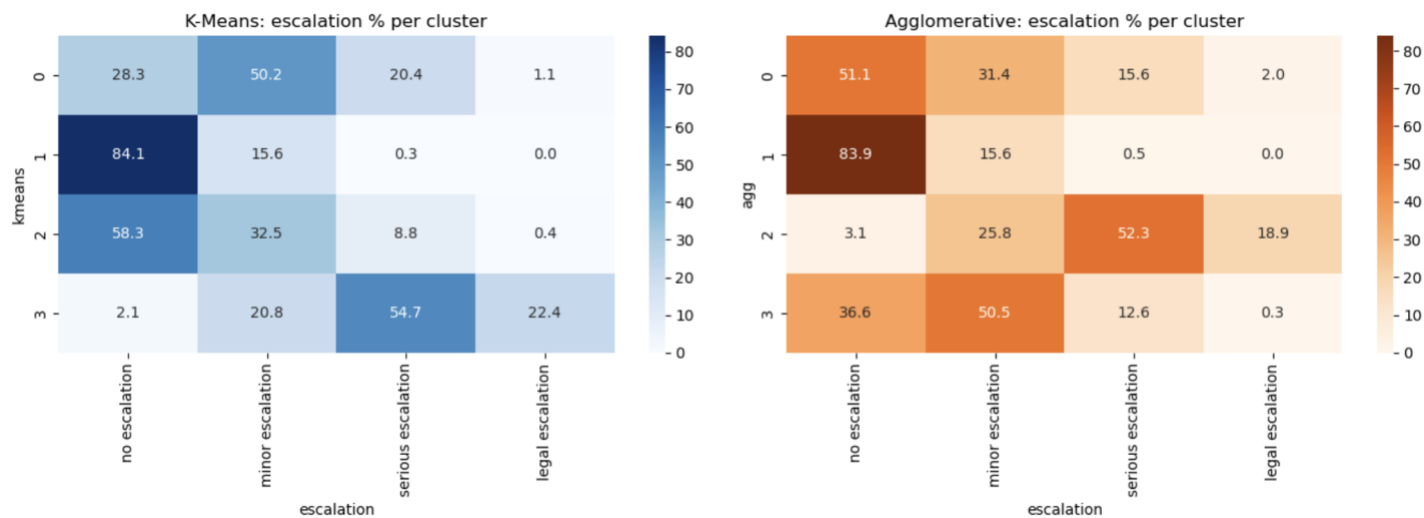


FIGURE 15: VISUALISING CLUSTER TO ESCALATION LEVEL

7. Key Findings

The major key findings from the analysis are:

- Escalation is rare but serious:** Nearly half of all interactions (49%) result in no escalation at all. Only 4% reach legal escalation. This means things are being handled fine day-to-day, but that small percentage of legal cases are costly, damaging, and should be a priority to prevent.
- First contact resolution is the norm:** Data exploration confirmed that 72.2% of interactions were resolved on first contact, with the remaining 27.8% flagged as unresolved, pointing to resolution quality as key for reducing escalation risk.
- The data can predict escalation with 96% accuracy:** The Gradient Boosting model correctly identified the escalation level 96% of the time. This is very high and means the patterns in the data are strong. All the information needed to predict who is about to escalate before it happens is present.
- The top warning signs for escalation:** From the SHAP analysis, the features that matter most shift slightly depending on the outcome being predicted, but the clearest signals across serious and legal escalations are risk indicator, service pressure score, customer engagement score, wait time in seconds, total burden, and sentiment score, with risk indicator and service pressure score consistently sitting at or near the top.
- There is a distinct group of high-risk customers:** The clustering (without being told labels) naturally found a group, K-Means cluster 3, that was 55% serious escalations and 22% legal escalations. This group exists in the data independently. These customers can be identified and targeted prior to escalating further.
- Most customers who escalate have a history:** High lifetime tickets, multiple SLA breaches, and frequent recent interactions are all connected to escalation. Escalating customers seem to have repeated bad experiences building up over time.

In conclusion, the dataset of 12,362 records required substantial cleaning, with misspelled categorical values, duplicate rows, and meaningful missingness across key behavioural fields all addressed before any modelling could take place. The supervised classification results were strong. Gradient Boosting emerged as the best performing model, achieving 96.3% accuracy, a macro F1 of 95.6%, and a ROC-AUC of 99.7%, outperforming Logistic Regression, Decision Tree, and Random Forest across every metric. The fact that even a baseline Logistic Regression model reached 95.5% accuracy suggests linear signal, validating the feature engineering choices made during preprocessing. SHAP analysis identified risk indicator, service pressure score, and customer engagement score as the most consistent influences of serious and legal escalation outcomes. The unsupervised clustering analysis reinforced these findings from a different angle. Without being given any labels, K-Means naturally isolated a cluster in which 55% of customers had experienced serious escalation and a further 22% had reached legal escalation. This confirms that a high-risk customer profile exists structurally within the data and can be detected independently of any model predictions.

Taken together, the findings demonstrate that escalation is not random. It follows identifiable patterns tied to service quality, customer history, and interaction outcomes. The tools developed here provide a foundation for early intervention, allowing attention toward the customers at most risk before situations become severe and costly.

Setup and Imports

This section imports all required libraries for data analysis, loads the dataset into a DataFrame, and defines a utility function that logs completed tasks to a separate .txt file for documentation and reproducibility.

```
In [1]: #Import libraries
import pandas as pd
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt

In [2]: # Function to record tasks into Travel_Log.txt
from datetime import datetime

def tlog(task_completed, why, tools_used="", notes="", file="travel_log.txt"):
    ts = datetime.now().astimezone().strftime("%Y-%m-%d %H:%M")
    entry = (
        f"[{ts}]\n"
        f"Task completed: {task_completed}\n"
        f"Why I did it: {why}\n"
        f"Tools/commands used (optional): {tools_used}\n"
        f"Notes: {notes}\n\n"
    )
    with open(file, "a", encoding="utf-8") as f:
        f.write(entry)

In [3]: # Import and read csv file
df = pd.read_csv("customer_service_escalation_risk.csv")
```

Data Exploration, Visualisations, and Summary

- Prepare an overview of the dataset, summarising the key features and their characteristics.
- Explore the structure and distribution of variables using appropriate visual and statistical methods.
- Introduce the dataset clearly in report, noting any interesting patterns or potential data quality concerns.

```
In [4]: df.sample(10) #Random sample to inspect data

Out [4]:
```

	escalation_level	age	tenure_months	loyalty_status	segment	region	industry	channel	issue_category	issue_subtype	...	behaviour_score_5	ops_telemetry_kpi_1	ops_telemetry_kpi_2	ops_telemetry_kpi_3	ops_telemetry_kpi_4	ops_telemetry_kpi_5	post_interaction_quality_score
3786	No escalation	37.0	40	Silver	Other	London	Healthcare	Chat	Technical	Delayed delivery	...	43.2	0.3449	-0.7922	-0.8878	-0.7321	1.2279	0.783
1637	No escalation	35.0	92	Silver	Retail	Midlands	Tech	Phone	Account	Locked account	...	72.7	0.6267	1.5089	2.1483	0.5920	-0.9493	-0.529
1946	Serious escalation	36.0	45	Gold	Retail	Midlands	Tech	Chat	Billing	Refund status	...	37.1	0.4621	0.9387	-0.7142	-0.2810	-1.0152	5.913
4113	No escalation	27.0	48	Bronze	Enterprise	London	Education	Chat	Returns	Refund status	...	67.6	-0.2503	-0.5260	-0.1560	-0.5306	-1.8246	0.703
7075	Minor escalation	40.0	81	Silver	Retail	Wales	Tech	Chat	Billing	Outage	...	37.7	-0.5774	-0.6966	0.1481	0.1203	-0.3267	3.520
10823	No escalation	29.0	49	Silver	Enterprise	South East	Education	Chat	Account	Password reset	...	57.6	-0.5554	-0.8602	1.3720	0.4900	-0.2552	-0.513
960	Legal escalation	24.0	38	Bronze	Enterprise	North West	Tech	Chat	Account	Locked account	...	53.1	-1.2216	0.3885	-0.2441	-1.3066	0.8157	8.529
9565	Minor escalation	43.0	23	Bronze	Business	North West	Finance	Phone	Billing	Payment failed	...	68.0	1.1436	-1.1837	0.4287	0.6298	0.4991	4.019
9634	Minor escalation	67.0	0	Bronze	Retail	London	Retail	Phone	Billing	Slow internet	...	42.9	-0.6149	0.5269	-0.1655	-0.2913	0.3581	2.056
6848	No escalation	45.0	42	Bronze	Enterprise	South East	Hospitality	Phone	Orders	Other	...	59.4	0.8453	0.2728	0.0940	-1.0219	-1.6290	0.462

10 rows x 41 columns

```
In [5]: df.info() #Basic information including data types and NaNs
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12362 entries, 0 to 12361
Data columns (total 41 columns):
#   Column                                Non-Null Count  Dtype
---  -
0    escalation_level                      12362 non-null  object
1    age                                  12331 non-null  float64
2    tenure_months                       12362 non-null  int64
3    loyalty_status                      12343 non-null  object
4    segment                             12362 non-null  object
5    region                              12362 non-null  object
6    industry                            12362 non-null  object
7    channel                             12362 non-null  object
8    issue_category                      12362 non-null  object
9    issue_subtype                      12362 non-null  object
10   product_line                        12362 non-null  object
11   device_type                        12362 non-null  object
12   time_of_day_bucket                 12346 non-null  object
13   wait_time_seconds                  12362 non-null  int64
14   previous_interactions_30d          12362 non-null  int64
15   resolution_time_minutes             12362 non-null  int64
16   agent_experience_years               12140 non-null  float64
17   first_contact_resolution            12362 non-null  object
18   number_of_transfers                 12362 non-null  int64
19   SLA_breaches_last_6m               12362 non-null  int64
20   lifetime_tickets                   12362 non-null  int64
21   average_response_delay              12362 non-null  float64
22   sentiment_score                     11732 non-null  float64
23   monthly_fee                         12362 non-null  float64
24   subscription_value_str              11963 non-null  object
25   churn_risk_band                     12362 non-null  object
26   priority_flag                       12362 non-null  object
27   behaviour_score_1                   12362 non-null  float64
28   behaviour_score_2                   12362 non-null  float64
29   behaviour_score_3                   12362 non-null  float64
30   behaviour_score_4                   12362 non-null  float64
31   behaviour_score_5                   12362 non-null  float64
32   ops_telemetry_kpi_1                 12362 non-null  float64
33   ops_telemetry_kpi_2                 12362 non-null  float64
34   ops_telemetry_kpi_3                 12362 non-null  float64
35   ops_telemetry_kpi_4                 12362 non-null  float64
36   ops_telemetry_kpi_5                 12362 non-null  float64
37   post_interaction_quality_score       12362 non-null  float64
38   risk_indicator                      12182 non-null  float64
39   service_pressure_score               12175 non-null  float64
40   customer_engagement_score           11536 non-null  float64
dtypes: float64(19), int64(7), object(15)
memory usage: 3.9+ MB
```

```
In [6]: df.shape #Check for number of rows/columns
```

Out[6]: (12362, 41)

```
In [7]: df[df.duplicated()] #Check for duplicate rows
```

Out[7]:

	escalation_level	age	tenure_months	loyalty_status	segment	region	industry	channel	issue_category	issue_subtype	...	behaviour_score_5	ops_telemetry_kpi_1	ops_telemetry_kpi_2	ops_telemetry_kpi_3	ops_telemetry_kpi_4	ops_telemetry_kpi_5	post_interaction_quality_sc
12240	No escalation	64.0	65	Silver	Retail	Wales	Education	Chat	Billing	Password reset	...	60.1	0.9054	-0.8847	1.2733	0.3218	-0.1293	-0.637
12241	No escalation	25.0	67	Gold	Retail	Midlands	Finance	Phone	Technical	Damaged item	...	51.0	2.4117	-0.6957	0.5792	1.4234	-0.1882	-0.370
12242	No escalation	23.0	17	Gold	Retail	Midlands	Finance	Chat	Billing	Outage	...	53.8	-0.3823	-0.9180	-0.2530	-1.4352	1.1338	0.505
12243	Minor escalation	39.0	51	Gold	Student	North West	Finance	Chat	Account	Delayed delivery	...	47.6	0.6831	0.1733	0.4972	-0.1135	0.2610	2.133
12244	Minor escalation	51.0	0	Bronze	Retail	South East	Retail	Phone	Orders	Outage	...	56.0	-0.0336	0.1786	-1.4131	-0.9482	0.1642	2.228
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
12357	Minor escalation	44.0	39	Silver	Retail	Midlands	Education	In-app	Orders	Invoice error	...	33.3	-1.2574	-1.6334	1.2038	0.5498	0.6220	3.517
12358	Legal escalation	51.0	100	Gold	Retail	London	Finance	Chat	Billing	Payment failed	...	66.7	0.0715	-1.9532	-0.1106	-0.0563	-1.8971	9.412
12359	No escalation	35.0	59	Bronze	Retail	North West	Tech	Phone	Technical	Other	...	43.2	-1.1632	0.0014	-0.6453	-0.8185	1.4606	0.771
12360	No escalation	83.0	0	Bronze	Enterprise	London	Hospitality	In-app	Account	Other	...	67.2	0.9176	-1.6990	0.3025	-0.9166	-0.3655	-0.225
12361	Minor escalation	35.0	23	Gold	Enterprise	Midlands	Education	Chat	Account	Other	...	34.4	-1.1123	0.5691	-0.7471	-0.3559	-0.8265	2.392

122 rows x 41 columns

```
In [8]: print(df.isnull().sum() * 100 / len(df)) #Check for missing values (Percentage)
```


escalation_level	0.000000
age	0.250768
tenure_months	0.000000
loyalty_status	0.153697
segment	0.000000
region	0.000000
industry	0.000000
channel	0.000000
issue_category	0.000000
issue_subtype	0.000000
product_line	0.000000
device_type	0.000000
time_of_day_bucket	0.129429
wait_time_seconds	0.000000
previous_interactions_30d	0.000000
resolution_time_minutes	0.000000
agent_experience_years	1.795826
first_contact_resolution	0.000000
number_of_transfers	0.000000
SLA_breaches_last_6m	0.000000
lifetime_tickets	0.000000
average_response_delay	0.000000
sentiment_score	5.096263
monthly_fee	0.000000
subscription_value_str	3.227633
churn_risk_band	0.000000
priority_flag	0.000000
behaviour_score_1	0.000000
behaviour_score_2	0.000000
behaviour_score_3	0.000000
behaviour_score_4	0.000000
behaviour_score_5	0.000000
ops_telemetry_kpi_1	0.000000
ops_telemetry_kpi_2	0.000000
ops_telemetry_kpi_3	0.000000
ops_telemetry_kpi_4	0.000000
ops_telemetry_kpi_5	0.000000
post_interaction_quality_score	0.000000
risk_indicator	1.456075
service_pressure_score	1.512700
customer_engagement_score	6.681767

dtype: float64

```
In [9]: df.nunique() #Check unique values in each column
```

escalation_level	4
age	74
tenure_months	126
loyalty_status	4
segment	5
region	47
industry	6
channel	4
issue_category	6
issue_subtype	106
product_line	4
device_type	4
time_of_day_bucket	4
wait_time_seconds	511
previous_interactions_30d	8
resolution_time_minutes	146
agent_experience_years	107
first_contact_resolution	2
number_of_transfers	6
SLA_breaches_last_6m	6
lifetime_tickets	82
average_response_delay	588
sentiment_score	11386
monthly_fee	5498
subscription_value_str	10535
churn_risk_band	3
priority_flag	2
behaviour_score_1	820
behaviour_score_2	849
behaviour_score_3	830
behaviour_score_4	855
behaviour_score_5	827
ops_telemetry_kpi_1	10249
ops_telemetry_kpi_2	10231
ops_telemetry_kpi_3	10201
ops_telemetry_kpi_4	10271
ops_telemetry_kpi_5	10202
post_interaction_quality_score	12000
risk_indicator	9104
service_pressure_score	11961
customer_engagement_score	10760

dtype: int64

```
In [10]: print (df['escalation_level'].value_counts(normalize=True) * 100) # Escalation level balance percentage

escalation_level
No escalation      48.883676
Minor escalation   30.440058
Serious escalation 16.712506
Legal escalation   3.963760
Name: proportion, dtype: float64

In [11]: df.describe() #Summary stats
```

Out[11]:

	age	tenure_months	wait_time_seconds	previous_interactions_30d	resolution_time_minutes	agent_experience_years	number_of_transfers	SLA_breaches_last_6m	lifetime_tickets	average_response_delay	...	behaviour_score_5	ops_telemetry_kpi_1	ops_telemetry_kpi_2	ops_telemetry_kpi_3
count	12331.000000	12362.000000	12362.000000	12362.000000	12362.000000	12140.000000	12362.000000	12362.000000	12362.000000	12362.000000	...	12362.000000	12362.000000	12362.000000	12362.000000
mean	39.422269	37.058809	184.20741	1.201424	42.750283	3.564629	0.608801	0.405193	15.903656	2.405965	...	50.028580	-0.008051	-0.002082	-0.002082
std	14.999103	25.404541	104.74980	1.097189	26.365153	2.093282	0.781747	0.631879	11.336211	1.181518	...	14.872114	1.004418	0.992245	0.992245
min	18.000000	0.000000	0.00000	0.000000	1.000000	0.000000	0.000000	0.000000	1.000000	0.000000	...	0.000000	-3.749600	-3.716000	-3.716000
25%	31.000000	17.000000	112.00000	0.000000	24.000000	2.100000	0.000000	0.000000	8.000000	1.570000	...	39.700000	-0.670900	-0.680925	-0.680925
50%	39.000000	36.000000	181.00000	1.000000	42.000000	3.500000	0.000000	0.000000	13.000000	2.390000	...	49.900000	-0.002100	-0.007200	-0.007200
75%	47.000000	54.000000	249.00000	2.000000	59.000000	5.000000	1.000000	1.000000	21.000000	3.210000	...	60.200000	0.666400	0.672175	0.672175
max	663.000000	141.000000	1520.00000	7.000000	470.000000	12.000000	5.000000	5.000000	93.000000	7.270000	...	100.000000	4.093700	3.385300	3.385300

8 rows × 26 columns

Figure 1

```
In [12]: sns.countplot(x='escalation_level', data=df, # Numeric class distribution of escalation level
                    order=['No escalation', 'Minor escalation', 'Serious escalation', 'Legal escalation'])
plt.tick_params(axis='x', rotation=15)
plt.title('Escalation level distribution')
plt.show()
```

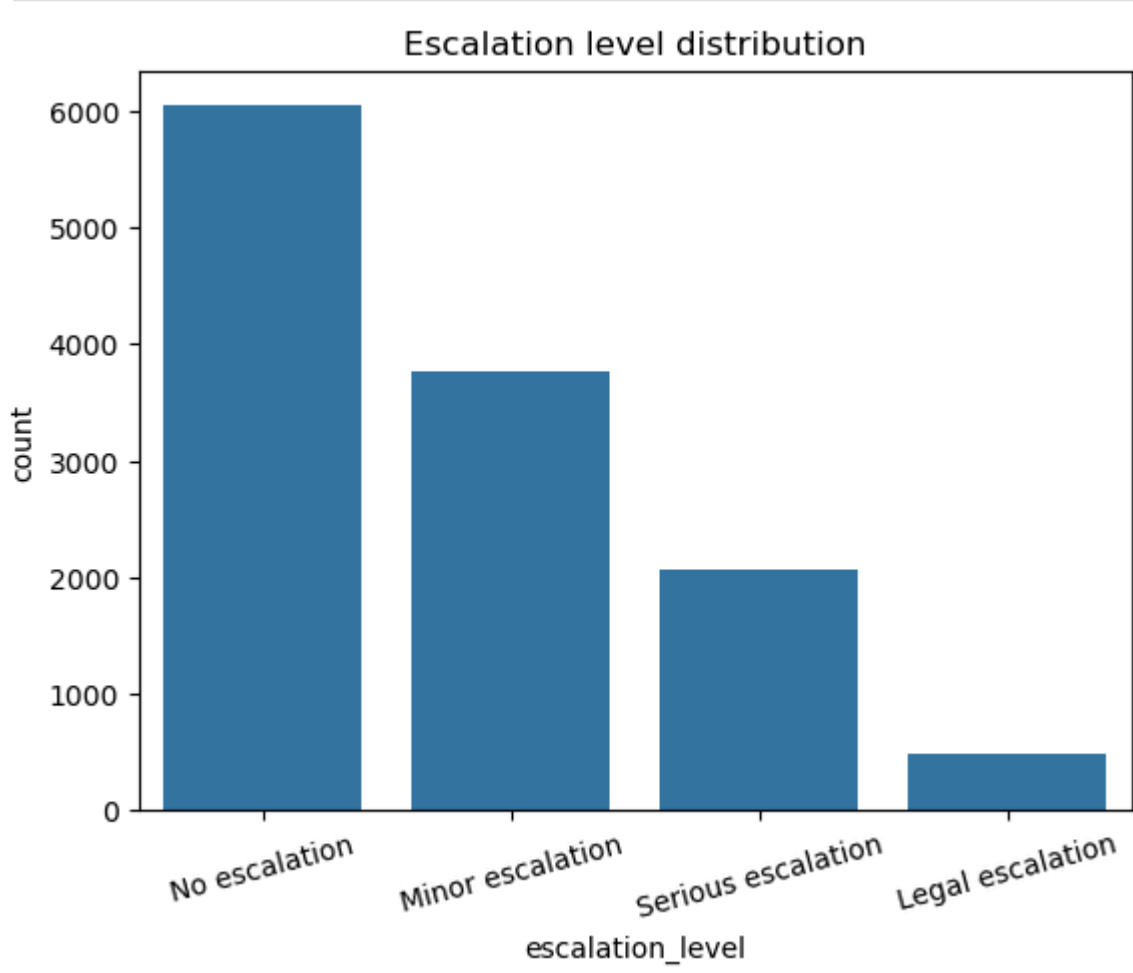


Figure 2

```
In [13]: import missingno as msno # AI suggested library
msno.heatmap(df, figsize=(12, 8), fontsize=10) # Show patterns (missingness correlation across columns)
plt.show()
```

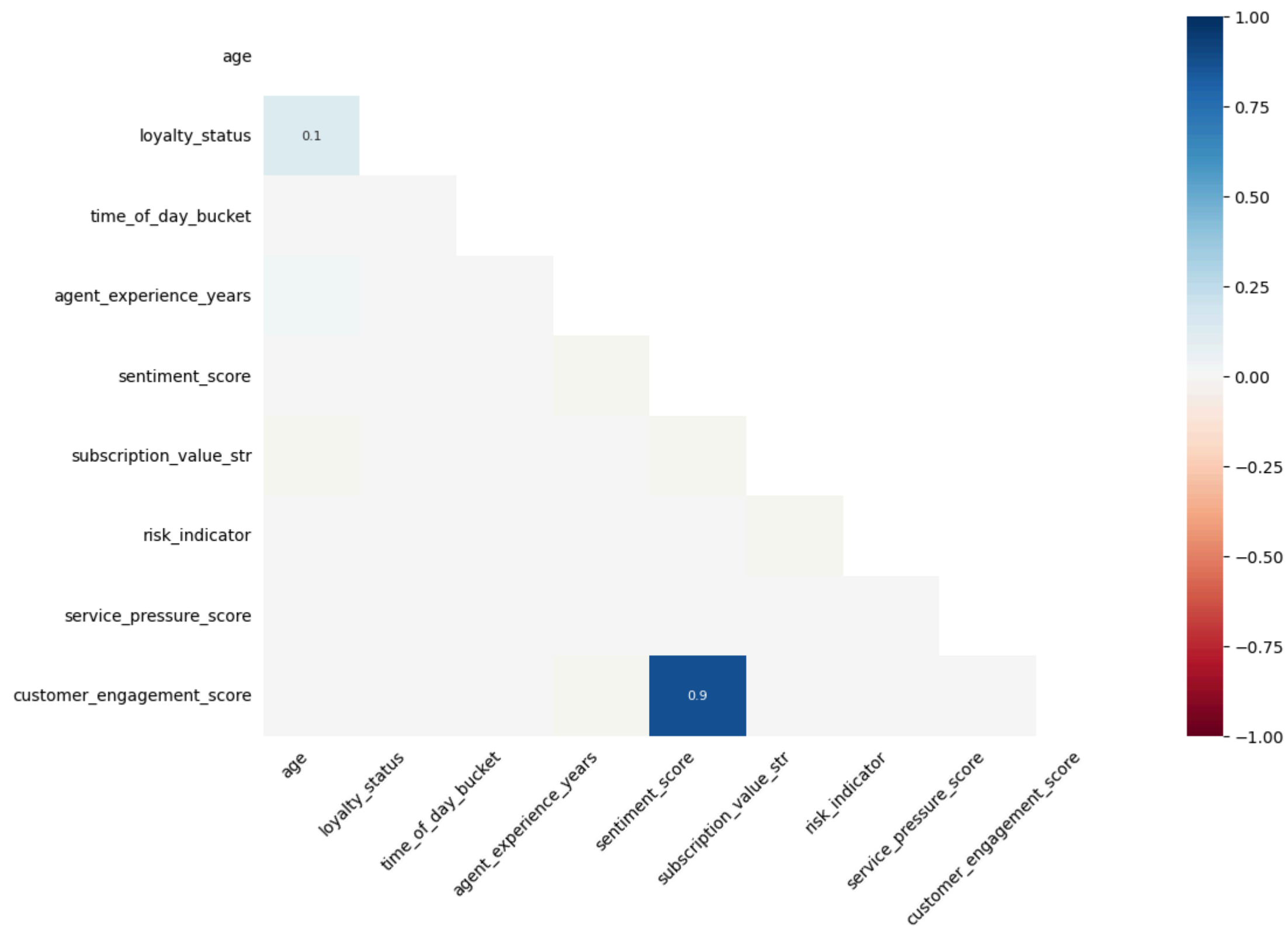



Figure 3

```
In [14]: fig, axes = plt.subplots(2, 3, figsize=(18, 10)) # Escalation level vs some numeric features (boxplots) AI retrieved

for ax, col in zip(axes.flatten(), ['wait_time_seconds', 'number_of_transfers', 'monthly_fee',
                                     'agent_experience_years', 'SLA_breaches_last_6m']):
    sns.boxplot(x='escalation_level', y=col, data=df, ax=ax,
               order=['No escalation', 'Minor escalation', 'Serious escalation', 'Legal escalation'])
    ax.set_title(col)
    ax.tick_params(axis='x', rotation=15)

axes[1, 2].set_visible(False)

plt.tight_layout()
plt.show()
```

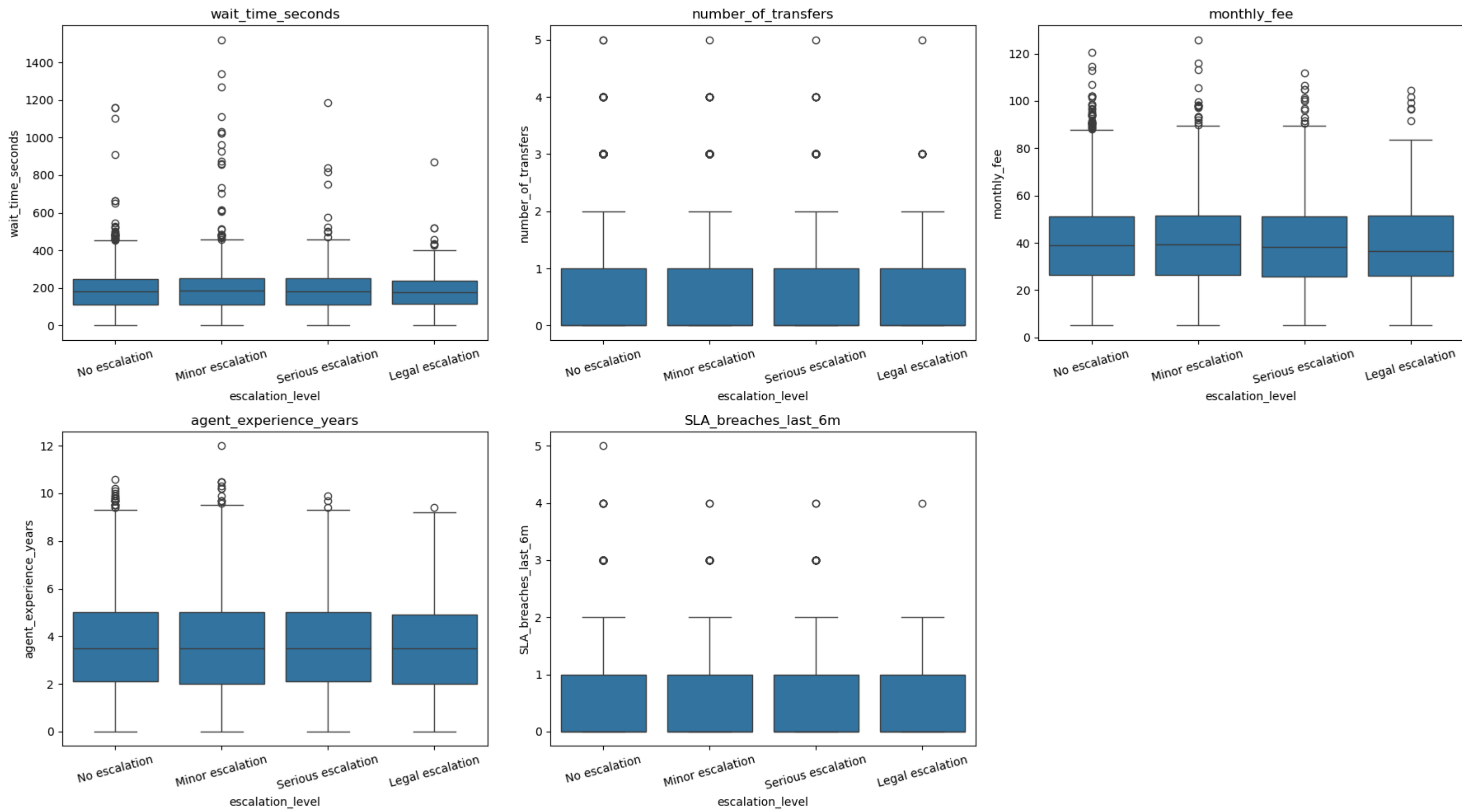


Figure 4

```
In [15]: fig, axes = plt.subplots(1, 2, figsize=(14, 5)) #Sentiment and engagement vs escalation level

for ax, col in zip(axes, ['sentiment_score', 'customer_engagement_score']):
    sns.boxplot(x='escalation_level', y=col, data=df, ax=ax, order=['No escalation', 'Minor escalation', 'Serious escalation', 'Legal escalation'])
    ax.set_title(col)
    ax.tick_params(axis='x', rotation=15)

plt.tight_layout()
plt.show()
```

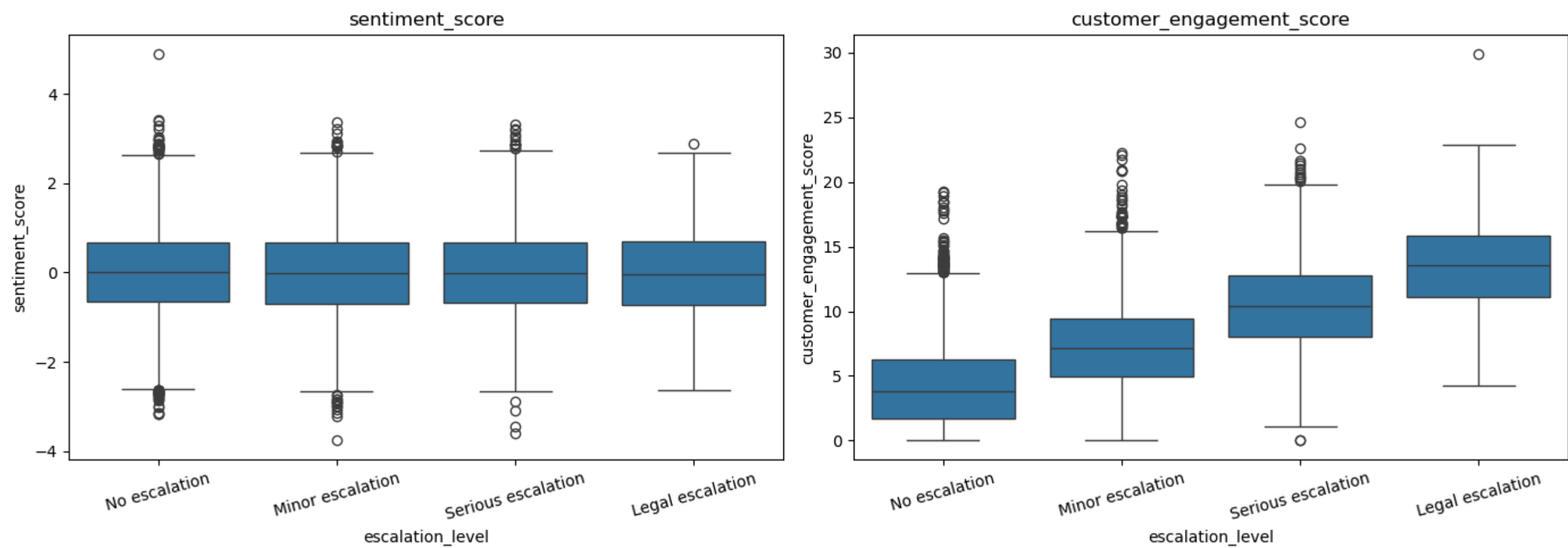
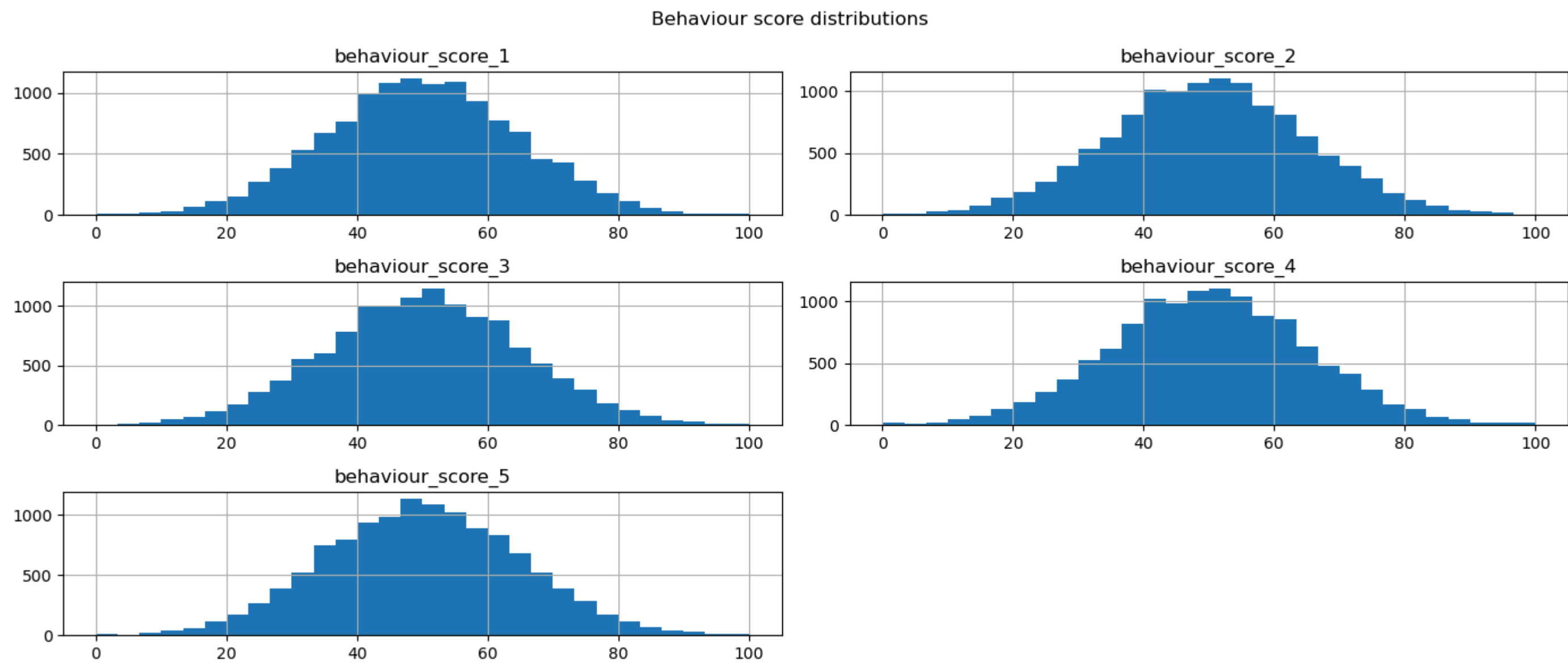


Figure 5

```
In [16]: # Behaviour scores distribution (Check if these are all on the same scale)
behaviour_cols = ['behaviour_score_1', 'behaviour_score_2', 'behaviour_score_3',
                  'behaviour_score_4', 'behaviour_score_5']

df[behaviour_cols].hist(figsize=(14, 6), bins=30)
plt.suptitle('Behaviour score distributions')
plt.tight_layout()
plt.show()
```



Data Dictionary (Raw)

This section includes initial data dictionary construction. Final data dictionary will be appended.

```
In [17]: data_dict = pd.DataFrame({
'column': df.columns,
'dtype': df.dtypes.values,
'non_null': df.notnull().sum().values,
```



```
'null_count': df.isnull().sum().values,
'null_pct': (df.isnull().mean() * 100).round(2).values,
'unique_values': df.nunique().values
})
data_dict
```

Out[17]:

	column	dtype	non_null	null_count	null_pct	unique_values
0	escalation_level	object	12362	0	0.00	4
1	age	float64	12331	31	0.25	74
2	tenure_months	int64	12362	0	0.00	126
3	loyalty_status	object	12343	19	0.15	4
4	segment	object	12362	0	0.00	5
5	region	object	12362	0	0.00	47
6	industry	object	12362	0	0.00	6
7	channel	object	12362	0	0.00	4
8	issue_category	object	12362	0	0.00	6
9	issue_subtype	object	12362	0	0.00	106
10	product_line	object	12362	0	0.00	4
11	device_type	object	12362	0	0.00	4
12	time_of_day_bucket	object	12346	16	0.13	4
13	wait_time_seconds	int64	12362	0	0.00	511
14	previous_interactions_30d	int64	12362	0	0.00	8
15	resolution_time_minutes	int64	12362	0	0.00	146
16	agent_experience_years	float64	12140	222	1.80	107
17	first_contact_resolution	object	12362	0	0.00	2
18	number_of_transfers	int64	12362	0	0.00	6
19	SLA_breaches_last_6m	int64	12362	0	0.00	6
20	lifetime_tickets	int64	12362	0	0.00	82
21	average_response_delay	float64	12362	0	0.00	588
22	sentiment_score	float64	11732	630	5.10	11386
23	monthly_fee	float64	12362	0	0.00	5498
24	subscription_value_str	object	11963	399	3.23	10535
25	churn_risk_band	object	12362	0	0.00	3
26	priority_flag	object	12362	0	0.00	2
27	behaviour_score_1	float64	12362	0	0.00	820
28	behaviour_score_2	float64	12362	0	0.00	849
29	behaviour_score_3	float64	12362	0	0.00	830
30	behaviour_score_4	float64	12362	0	0.00	855
31	behaviour_score_5	float64	12362	0	0.00	827
32	ops_telemetry_kpi_1	float64	12362	0	0.00	10249
33	ops_telemetry_kpi_2	float64	12362	0	0.00	10231
34	ops_telemetry_kpi_3	float64	12362	0	0.00	10201
35	ops_telemetry_kpi_4	float64	12362	0	0.00	10271
36	ops_telemetry_kpi_5	float64	12362	0	0.00	10202
37	post_interaction_quality_score	float64	12362	0	0.00	12000
38	risk_indicator	float64	12182	180	1.46	9104
39	service_pressure_score	float64	12175	187	1.51	11961
40	customer_engagement_score	float64	11536	826	6.68	10760

Data Cleansing, Feature Engineering, and Pre-processing

- Apply any cleansing, transformation, or pre-processing operations.
- Identify variables that may be unsuitable for prediction due to meaning, timing, leakage risk, or reliability, and justify whether they should be retained.

Data Types

```
In [18]: # Converting all values to float
df['subscription_value_str'] = df['subscription_value_str'].astype(str).str.replace('[£,]', '', regex=True).astype(float)

df.rename(columns={'subscription_value_str': 'subscription_value'}, inplace=True) #Renamed

In [19]: df.dtypes['subscription_value'] # Check

Out[19]: dtype('float64')

In [20]: if df['first_contact_resolution'].dtype == 'object': # Convert to binary integer
          df['first_contact_resolution'] = df['first_contact_resolution'].map({'Yes': 1, 'No': 0})
          print(df['first_contact_resolution'].value_counts())

first_contact_resolution
1      8922
0     3440
Name: count, dtype: int64

In [21]: df['priority_flag'].value_counts() #Check if binary

Out[21]: priority_flag
No      10067
Yes      2295
Name: count, dtype: int64
```

String Cleaning

```
In [22]: for col in df.select_dtypes(include='object').columns: #Check for inconsistent spelling
          df[col] = df[col].str.strip().str.lower()

In [23]: for col in df.select_dtypes(include='object').columns: # Check spelling or near duplicates
          print(col, df[col].unique())

escalation_level ['minor escalation' 'no escalation' 'serious escalation'
                  'legal escalation']
loyalty_status  ['bronze' 'silver' 'gold' nan 'platinum']
segment         ['retail' 'business' 'enterprise' 'student' 'other']
region          ['london' 'south east' 'wales' 'scotland' 'north west' 'midlands' 'lodnon'
                  'sotuh east' 'olndon' 'midlnads' 'londno' 'imdlands' 'midladns'
                  'souht east' 'awles' 'midlansd' 'north wets' 'north ewst' 'suoth east'
                  'wael's' 'norht west' 'southe ast' 'northw est' 'osuth east' 'mid-lands'
                  'sout heast' 'mdilands' 'lonodn' 'soctland' 'notrh west' 'nroth west'
                  'south esat' 'south aest' 'lnodon' 'scotladn' 'mida'lnds' 'north wset'
                  'wlaes' 'south eats' 'onrth west' 'mildands' 'csotland' 'scotlnad'
                  'scoltand' 'walse' 'nort hwest' 'scotalnd']
industry        ['retail' 'finance' 'hospitality' 'tech' 'education' 'healthcare']
channel         ['chat' 'phone' 'email' 'in-app']
issue_category  ['technical' 'billing' 'returns' 'orders' 'other' 'account']
issue_subtype   ['wrong item' 'locked account' 'other' 'slow internet' 'invoice error'
                  'damaged item' 'refund status' 'outage' 'payment failed'
                  'delayed delivery' 'password reset' 'password reste' 'ohter'
                  'apssword reset' 'solw internet' 'ouatge' 'password reest' 'wron gitem'
                  'payment falied' 'passwodr reset' 'payment failde' 'lockeda ccount'
                  'uotage' 'wrongi tem' 'passowrd reset' 'invoic eerror' 'lckoked account'
                  'toher' 'damagedi tem' 'slo winternet' 'paymentf ailed' 'refund stauts'
                  'locke daccount' 'slow internte' 'payment fialed' 'otehr'
                  'password rseet' 'delayedd elivery' 'refund tsatus' 'damage ditem'
                  'outaeg' 'slow itnernet' 'wrnog item' 'dmaaged item' 'damaegd item'
                  'invocie error' 'inovice error' 'damaged tiem' 'delayed delievry'
                  'passwor dreset' 'wrong itme' 'damagde item' 'invoiec error'
                  'slow inetrnet' 'payment faield' 'locked cacount' 'password erset'
                  'invoice erro' 'slwo internet' 'paymen tfailed' 'refund sattus'
                  'olcked account' 'damaged ietm' 'slowi nternet' 'delayed delivrey'
                  'refund sttaus' 'slow intrenet' 'pyament failed' 'rfeund status'
                  'lokced account' 'refnud status' 'delaeyd delivery' 'dleayed delivery'
                  'refun dstatus' 'psasword reset' 'passwordr eset' 'slow interent'
                  'refudn status' 'rwong item' 'invoice errorr' 'edlayed delivery'
                  'refund statsu' 'pamyent failed' 'locked accuont' 'dealyed delivery'
                  'wornng item' 'delayde delivery' 'damaged itme' 'wrong ietm'
                  'delayed deilvery' 'delayed edlivery' 'othre' 'lsow internet'
                  'delayed deliveyr' 'lockde account' 'outgae' 'erfund status'
                  'damgaed item' 'payment afiled' 'delyaed delivery' 'ivnoice error'
                  'delayed delviery' 'reufnd status' 'slow intenret' 'wrogn item'
                  'daamged item']
product_line    ['enterprise' 'standard' 'basic' 'premium']
device_type     ['mobile' 'call-centre' 'desktop' 'web']
time_of_day_bucket ['evening' 'afternoon' 'morning' 'night' nan]
churn_risk_band ['low' 'medium' 'high']
priority_flag    ['no' 'yes']

In [24]: from thefuzz import process # AI retrieved (installed pip install thefuzz)
          # Correct region/issue spelling into variable
```

```
valid_regions = ['south east', 'midlands', 'london', 'north west', 'scotland', 'wales']
valid_issue_subtype = ['other', 'slow internet', 'delayed delivery', 'damaged item', 'wrong item',
                       'password reset', 'invoice error', 'outage', 'refund status',
                       'locked account', 'payment failed']

#Function that fixes typos in a column by finding the closest correct spelling
def standardise_column(df, col, valid_categories):
    def fix(val):
        if pd.isnull(val):
            return val
        match, score = process.extractOne(val, valid_categories)
        return match if score >= 80 else val
    df[col] = df[col].apply(fix)
    return df

df = standardise_column(df, 'region', valid_regions)
df = standardise_column(df, 'issue_subtype', valid_issue_subtype)

# Verify
print(df['region'].value_counts())
print(df['issue_subtype'].value_counts())
```

```
region
south east    2919
midlands      2586
london        2556
north west    1845
scotland      1455
wales         1001
Name: count, dtype: int64
issue_subtype
other          1174
slow internet  1143
damaged item   1139
delayed delivery 1136
wrong item     1131
outage         1124
password reset  1119
invoice error   1111
refund status  1102
locked account  1098
payment failed  1085
Name: count, dtype: int64
```

```
In [25]: # Check for duplicates after string cleaning
print(f'Duplicates before: {df.duplicated().sum()}')
```

Duplicates before: 122

```
In [26]: df.drop_duplicates(inplace=True) #Drop duplicates
print(f'Duplicates after: {df.duplicated().sum()}')
```

Duplicates after: 0

Outliers

```
In [27]: print(df[df['age'] > 120]['age'].value_counts()) #Check age column outliers
```

```
age
245.0    1
630.0    1
155.0    1
663.0    1
399.0    1
440.0    1
228.0    1
Name: count, dtype: int64
```

```
In [28]: df.loc[df['age'] > 120, 'age'] = np.nan #Replace unreasonable values with NaNs and impute w/ median
df['age'] = df['age'].fillna(df['age'].median())
```

```
In [29]: def flag_outliers(df, col): #Check numeric columns outliers using IQR
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return df[(df[col] < lower) | (df[col] > upper)].shape[0]

numeric_cols = df.select_dtypes(include='number').columns
for col in numeric_cols:
    n = flag_outliers(df, col)
    if n > 0:
        print(f'{col}: {n} outliers')
```


age: 43 outliers
tenure_months: 40 outliers
wait_time_seconds: 71 outliers
previous_interactions_30d: 21 outliers
resolution_time_minutes: 65 outliers
agent_experience_years: 41 outliers
number_of_transfers: 303 outliers
SLA_breaches_last_6m: 91 outliers
lifetime_tickets: 311 outliers
average_response_delay: 45 outliers
sentiment_score: 90 outliers
monthly_fee: 70 outliers
subscription_value: 41 outliers
behaviour_score_1: 95 outliers
behaviour_score_2: 94 outliers
behaviour_score_3: 78 outliers
behaviour_score_4: 104 outliers
behaviour_score_5: 70 outliers
ops_telemetry_kpi_1: 97 outliers
ops_telemetry_kpi_2: 68 outliers
ops_telemetry_kpi_3: 78 outliers
ops_telemetry_kpi_4: 85 outliers
ops_telemetry_kpi_5: 84 outliers
post_interaction_quality_score: 184 outliers
risk_indicator: 107 outliers
service_pressure_score: 83 outliers
customer_engagement_score: 99 outliers

```
In [30]: def cap_outliers(df, col):  
        Q1 = df[col].quantile(0.25)  
        Q3 = df[col].quantile(0.75)  
        IQR = Q3 - Q1  
        lower = Q1 - 1.5 * IQR  
        upper = Q3 + 1.5 * IQR  
        df[col] = df[col].clip(lower=lower, upper=upper)  
        return df
```

```
In [31]: cols_to_cap = [ #Cap extreme values  
                        'wait_time_seconds', 'resolution_time_minutes', 'average_response_delay',  
                        'number_of_transfers', 'SLA_breaches_last_6m', 'lifetime_tickets',  
                        'previous_interactions_30d', 'monthly_fee', 'subscription_value',  
                        'sentiment_score', 'customer_engagement_score', 'risk_indicator',  
                        'service_pressure_score', 'behaviour_score_1', 'behaviour_score_2',  
                        'behaviour_score_3', 'behaviour_score_4', 'behaviour_score_5',  
                        'ops_telemetry_kpi_1', 'ops_telemetry_kpi_2', 'ops_telemetry_kpi_3',  
                        'ops_telemetry_kpi_4', 'ops_telemetry_kpi_5']  
  
        for col in cols_to_cap:  
            df = cap_outliers(df, col)
```

```
In [32]: print(df[cols_to_cap].describe().loc[['min', 'max']]) #Check
```

	wait_time_seconds	resolution_time_minutes	average_response_delay	\
min	0.0	1.0	0.00	
max	454.5	111.5	5.67	
	number_of_transfers	SLA_breaches_last_6m	lifetime_tickets	\
min	0.0	0.0	1.0	
max	2.5	2.5	43.0	
	previous_interactions_30d	monthly_fee	subscription_value	\
min	0.0	5.000	50.000	
max	5.0	88.775	1080.625	
	sentiment_score	...	behaviour_score_1	behaviour_score_2 \
min	-2.682905	...	10.7	9.5
max	2.674903	...	89.1	90.3
	behaviour_score_3	behaviour_score_4	behaviour_score_5	\
min	9.9	9.6	8.95	
max	90.7	90.4	90.95	
	ops_telemetry_kpi_1	ops_telemetry_kpi_2	ops_telemetry_kpi_3	\
min	-2.673875	-2.702413	-2.708438	
max	2.671325	2.698688	2.713262	
	ops_telemetry_kpi_4	ops_telemetry_kpi_5		
min	-2.735113	-2.713413		
max	2.716588	2.710888		
[2 rows x 23 columns]				

```
In [33]: print(df['age'].isna().sum()) # Check for NaNs  
        print(df['age'].describe()) # Confirm max
```

```
0
count      12240.000000
mean        39.216422
std         11.566162
min         18.000000
25%         31.000000
50%         39.000000
75%         47.000000
max         90.000000
Name: age, dtype: float64
```

Missing Values

```
In [34]: #Created flag before imputing
df['sentiment_engagement_missing'] = (
    df['sentiment_score'].isnull() | df['customer_engagement_score'].isnull()
).astype(int)
```

```
In [35]: numeric_to_impute = ['age', 'agent_experience_years', 'risk_indicator',
                             'service_pressure_score', 'subscription_value']

for col in numeric_to_impute: # Numeric imputation
    df[col] = df[col].fillna(df[col].median())

categorical_to_impute = ['loyalty_status', 'time_of_day_bucket']

for col in categorical_to_impute: # Categorical imputation
    df[col] = df[col].fillna(df[col].mode()[0])

# Impute sentiment and engagement with median
df['sentiment_score'] = df['sentiment_score'].fillna(df['sentiment_score'].median())
df['customer_engagement_score'] = df['customer_engagement_score'].fillna(df['customer_engagement_score'].median())
```

```
In [36]: # Verify no nulls remain
print(df.isnull().sum()[df.isnull().sum() > 0])
print(df['sentiment_engagement_missing'].value_counts())

Series([], dtype: int64)
sentiment_engagement_missing
0      11422
1         818
Name: count, dtype: int64
```

Leakage Risks

```
In [37]: #Dropping columns
df.drop(columns=['post_interaction_quality_score', 'resolution_time_minutes', 'first_contact_resolution'], inplace=True)
```

```
In [38]: print(df.shape) #Check if columns dropped
print(df.columns.tolist())

(12240, 39)
['escalation_level', 'age', 'tenure_months', 'loyalty_status', 'segment', 'region', 'industry', 'channel', 'issue_category', 'issue_subtype', 'product_line', 'device_type', 'time_of_day_bucket', 'wait_time_seconds', 'previous_interactions_30d', 'agent_experience_years', 'number_of_transfers', 'SLA_breaches_last_6m', 'lifetime_tickets', 'average_response_delay', 'sentiment_score', 'monthly_fee', 'subscription_value', 'churn_risk_band', 'priority_flag', 'behaviour_score_1', 'behaviour_score_2', 'behaviour_score_3', 'behaviour_score_4', 'behaviour_score_5', 'ops_telemetry_kpi_1', 'ops_telemetry_kpi_2', 'ops_telemetry_kpi_3', 'ops_telemetry_kpi_4', 'ops_telemetry_kpi_5', 'risk_indicator', 'service_pressure_score', 'customer_engagement_score', 'sentiment_engagement_missing']
```

Class Balance

```
In [39]: df['escalation_level'].value_counts(normalize=True) * 100 #Check target variable
```

```
Out[39]: escalation_level
no escalation      48.937908
minor escalation   30.416667
serious escalation  16.699346
legal escalation    3.946078
Name: proportion, dtype: float64
```

Feature Engineering

```
In [40]: # Total operational burden
df['total_burden'] = df['number_of_transfers'] + df['SLA_breaches_last_6m']

# High risk flag
df['high_risk'] = ((df['churn_risk_band'] == 'high') & (df['priority_flag'] == 1)).astype(int)

# Interaction history ratio
df['recent_interaction_ratio'] = df['previous_interactions_30d'] / (df['lifetime_tickets'] + 1)
```

Encode Categorials


```
In [41]: # Convert priority flag to binary
df['priority_flag'] = df['priority_flag'].map({'yes': 1, 'no': 0})
# Encode nominal categoricals
nominal_cols = ['channel', 'issue_category', 'issue_subtype', 'product_line',
                'device_type', 'region', 'industry', 'segment',
                'time_of_day_bucket', 'churn_risk_band', 'loyalty_status']
#AI and additional learning materials retrieved
df = pd.get_dummies(df, columns=nominal_cols, drop_first=True) #Missing column for features w/ many categories

# Label encode the target
from sklearn.preprocessing import LabelEncoder #Help from video https://youtu.be/YvEx0IGKTko?si=CIgoZRYHGpY8xXhv
le = LabelEncoder()
df['escalation_level_enc'] = le.fit_transform(df['escalation_level'])

# Verify
print(df.shape)
print(df.columns.tolist())
print(df['escalation_level_enc'].value_counts())
```

```
(12240, 78)
['escalation_level', 'age', 'tenure_months', 'wait_time_seconds', 'previous_interactions_30d', 'agent_experience_years', 'number_of_transfers', 'SLA_breaches_last_6m', 'lifetime_tickets', 'average_response_delay', 'sentiment_score', 'monthly_fee', 'subscripti
on_value', 'priority_flag', 'behaviour_score_1', 'behaviour_score_2', 'behaviour_score_3', 'behaviour_score_4', 'behaviour_score_5', 'ops_telemetry_kpi_1', 'ops_telemetry_kpi_2', 'ops_telemetry_kpi_3', 'ops_telemetry_kpi_4', 'ops_telemetry_kpi_5', 'risk_indic
ator', 'service_pressure_score', 'customer_engagement_score', 'sentiment_engagement_missing', 'total_burden', 'high_risk', 'recent_interaction_ratio', 'channel_email', 'channel_in-app', 'channel_phone', 'issue_category_billing', 'issue_category_orders', 'issu
e_category_other', 'issue_category_returns', 'issue_category_technical', 'issue_subtype_delayed delivery', 'issue_subtype_invoice error', 'issue_subtype_locked account', 'issue_subtype_other', 'issue_subtype_outage', 'issue_subtype_password reset', 'issue_sub
type_payment failed', 'issue_subtype_refund status', 'issue_subtype_slow internet', 'issue_subtype_wrong item', 'product_line_enterprise', 'product_line_premium', 'product_line_standard', 'device_type_desktop', 'device_type_mobile', 'device_type_web', 'region
_midlands', 'region_north west', 'region_scotland', 'region_south east', 'region_wales', 'industry_finance', 'industry_healthcare', 'industry_hospitality', 'industry_retail', 'industry_tech', 'segment_enterprise', 'segment_other', 'segment_retail', 'segment_s
tudent', 'time_of_day_bucket_evening', 'time_of_day_bucket_morning', 'time_of_day_bucket_night', 'churn_risk_band_low', 'churn_risk_band_medium', 'loyalty_status_gold', 'loyalty_status_platinum', 'loyalty_status_silver', 'escalation_level_enc']
escalation_level_enc
2      5990
1      3723
3      2044
0       483
Name: count, dtype: int64
```

```
In [42]: print(dict(zip(le.classes_, le.transform(le.classes_)))) #Verify the label encoding matches correctly

{'legal_escalation': np.int64(0), 'minor_escalation': np.int64(1), 'no_escalation': np.int64(2), 'serious_escalation': np.int64(3)}
```

```
In [43]: issue_subtype_cols = [col for col in df.columns if col.startswith('issue_subtype-')]
damaged_item_rows = df[(df[issue_subtype_cols] == 0).all(axis=1)]
print(len(damaged_item_rows)) #Check columns set to 0/rows where the issue subtype encoding is blank

1133
```

Train/Test Split and Scale

```
In [44]: #Help from video https://youtu.be/Sj0fbbfI2qY?si=WBBwbe8XzLNU-TgY
from sklearn.model_selection import train_test_split #Imports library
from sklearn.preprocessing import StandardScaler # Import the tool that scales

X = df.drop(columns=['escalation_level', 'escalation_level_enc']) # Every column except the target
y = df['escalation_level_enc'] #Just the target column

X_train, X_test, y_train, y_test = train_test_split( #Split both X and y into two groups
    X, y, test_size=0.2, stratify=y, random_state=42) # Keep 20% for testing, 80% goes to training
```

```
In [45]: scaler = StandardScaler() #Everything on the same scale
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(X_train.shape, X_test.shape)
print(y_train.value_counts(normalize=True).round(3))

(9792, 76) (2448, 76)
escalation_level_enc
2      0.489
1      0.304
3      0.167
0      0.040
Name: proportion, dtype: float64
```

```
In [46]: # Check sizes
print(f"Training rows: {X_train.shape[0]}")
print(f"Testing rows: {X_test.shape[0]}")
print(f"Total: {X_train.shape[0] + X_test.shape[0]}")
print(f"Features: {X_train.shape[1]}")

# Check stratification
print("\nTrain class distribution:")
print(y_train.value_counts(normalize=True).round(3))
```

Training rows: 9792
Testing rows: 2448
Total: 12240
Features: 76

Train class distribution:
escalation_level_enc
2 0.489
1 0.304
3 0.167
0 0.040
Name: proportion, dtype: float64

Supervised Model Training, and Tuning

- Develop predictive models using tools and libraries (e.g. k-NN, Decision Trees, SVMs, ensembles, neural networks, etc.).
- Perform reasonable hyperparameter tuning to improve model performances.
- Evaluate models using suitable metrics for a multiclass problem, including comparative analysis.
- Provide interpretation of model behaviour, including identification of influential factors or variables.

Logistic Regression (Baseline model, no tuning)

Figure 6

In [47]:

```
from sklearn.linear_model import LogisticRegression #Help from video https://youtu.be/aL21Y-u0SRs?si=9mb3Vnijy1CUExj0
from sklearn.metrics import classification_report, ConfusionMatrixDisplay

# Baseline Logistic Regression (no tuning, class weight to handle imbalance)
lr_baseline = LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)
lr_baseline.fit(X_train_scaled, y_train)

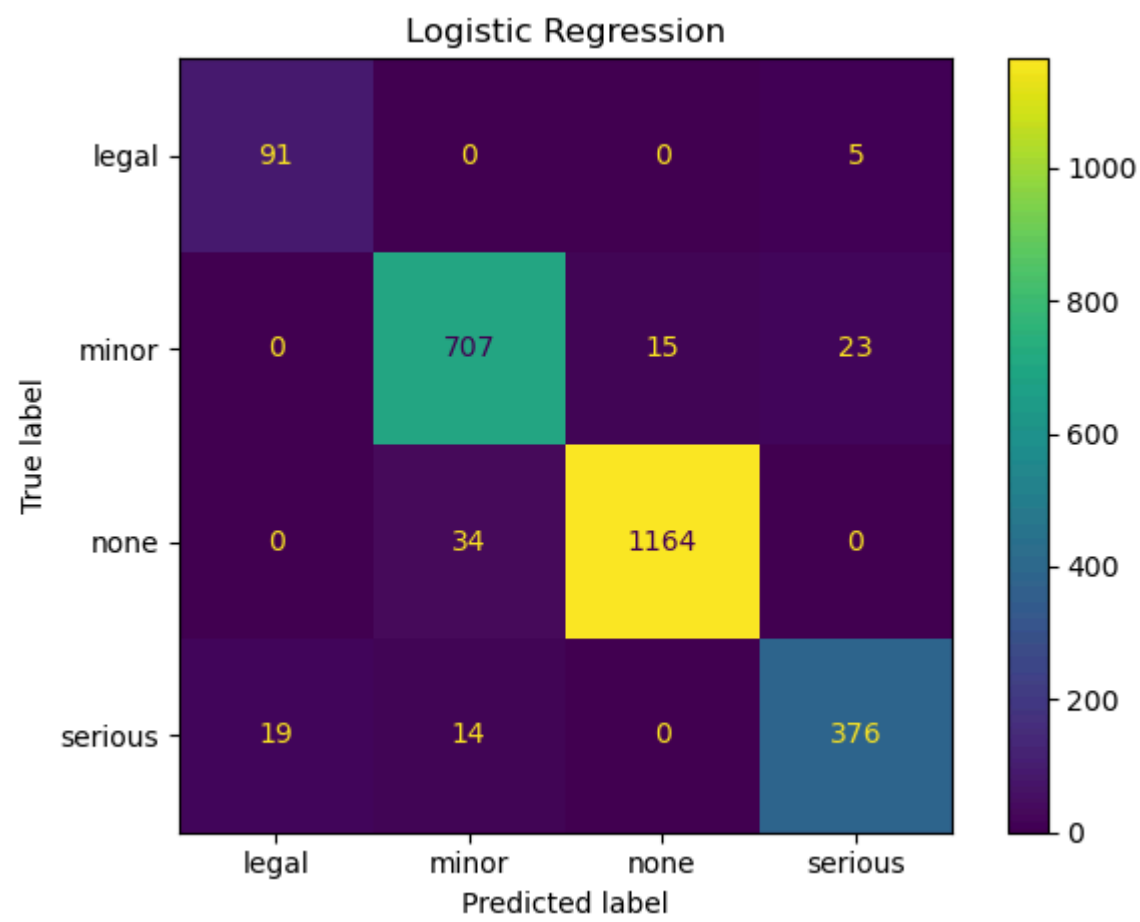
y_pred_lr = lr_baseline.predict(X_test_scaled)

print("=== Logistic Regression (Baseline) ===")
print(classification_report(y_test, y_pred_lr,
                           target_names=['legal escalation', 'minor escalation', 'no escalation', 'serious escalation']))

ConfusionMatrixDisplay.from_estimator(lr_baseline, X_test_scaled, y_test,
                                     display_labels=['legal', 'minor', 'none', 'serious'])
plt.title("Logistic Regression")
plt.tight_layout()
plt.show()
```

=== Logistic Regression (Baseline) ===

	precision	recall	f1-score	support
legal escalation	0.83	0.95	0.88	96
minor escalation	0.94	0.95	0.94	745
no escalation	0.99	0.97	0.98	1198
serious escalation	0.93	0.92	0.92	409
accuracy			0.96	2448
macro avg	0.92	0.95	0.93	2448
weighted avg	0.96	0.96	0.96	2448



Decision Tree (Tuned)

Figures 7 and 8

```
In [48]: from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV
# Help from video https://youtu.be/wxS5P7yDHRA?si=-9_mwZym0PfCdE6P
# Find best depth first
param_dist = {
    'max_depth': [3, 5, 7, 10, 15, 20, None],
    'min_samples_split': [2, 10, 20, 50],
    'min_samples_leaf': [1, 5, 10, 20],
    'criterion': ['gini', 'entropy']}

dt = DecisionTreeClassifier(class_weight='balanced', random_state=42)

dt_search = RandomizedSearchCV( #Help from video https://youtu.be/PPohAc07TSo?si=j58bpneeGyvFifaI
    dt, param_dist,
    n_iter=30,
    scoring='f1_macro',
    cv=5,
    random_state=42,
    n_jobs=-1,
    verbose=1)

dt_search.fit(X_train_scaled, y_train)

print(f"Best params: {dt_search.best_params_}")
print(f"Best CV macro F1: {dt_search.best_score_:.4f}")

# Evaluate best model
best_dt = dt_search.best_estimator_
y_pred_dt = best_dt.predict(X_test_scaled)

print("\n=== Decision Tree (Tuned) ===")
print(classification_report(y_test, y_pred_dt,
    target_names=['legal escalation', 'minor escalation', 'no escalation', 'serious escalation']))

ConfusionMatrixDisplay.from_estimator(best_dt, X_test_scaled, y_test,
    display_labels=['legal', 'minor', 'none', 'serious'])
plt.title("Decision Tree (Tuned)")
plt.tight_layout()
plt.show()

# Visualise tree
depths = range(2, 16)
train_scores, test_scores = [], []
for d in depths:
    dt_d = DecisionTreeClassifier(max_depth=d, class_weight='balanced', random_state=42)
    dt_d.fit(X_train_scaled, y_train)
    train_scores.append(dt_d.score(X_train_scaled, y_train))
    test_scores.append(dt_d.score(X_test_scaled, y_test))

plt.figure(figsize=(8, 4))
plt.plot(depths, train_scores, label='Train Accuracy')
plt.plot(depths, test_scores, label='Test Accuracy')
```

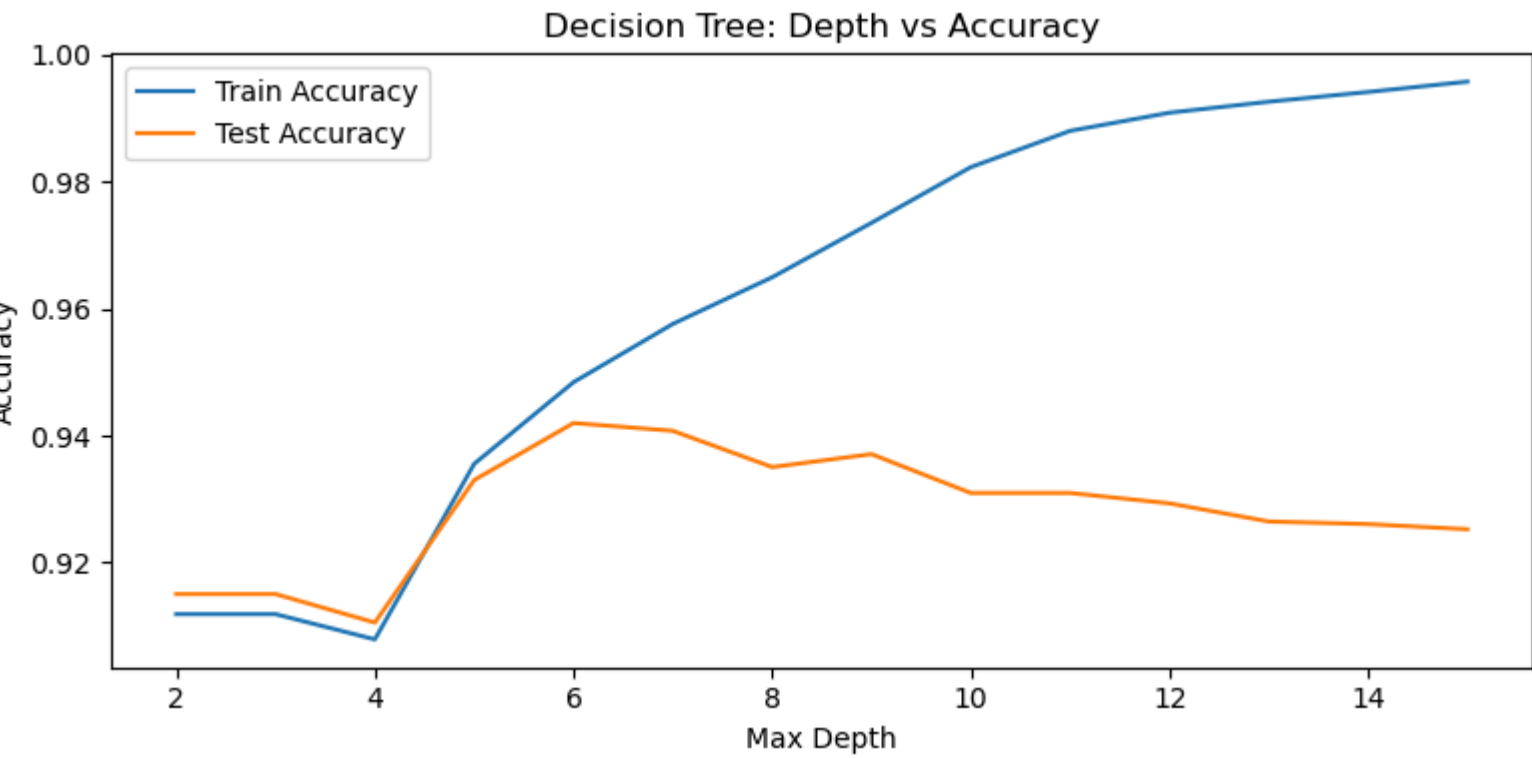
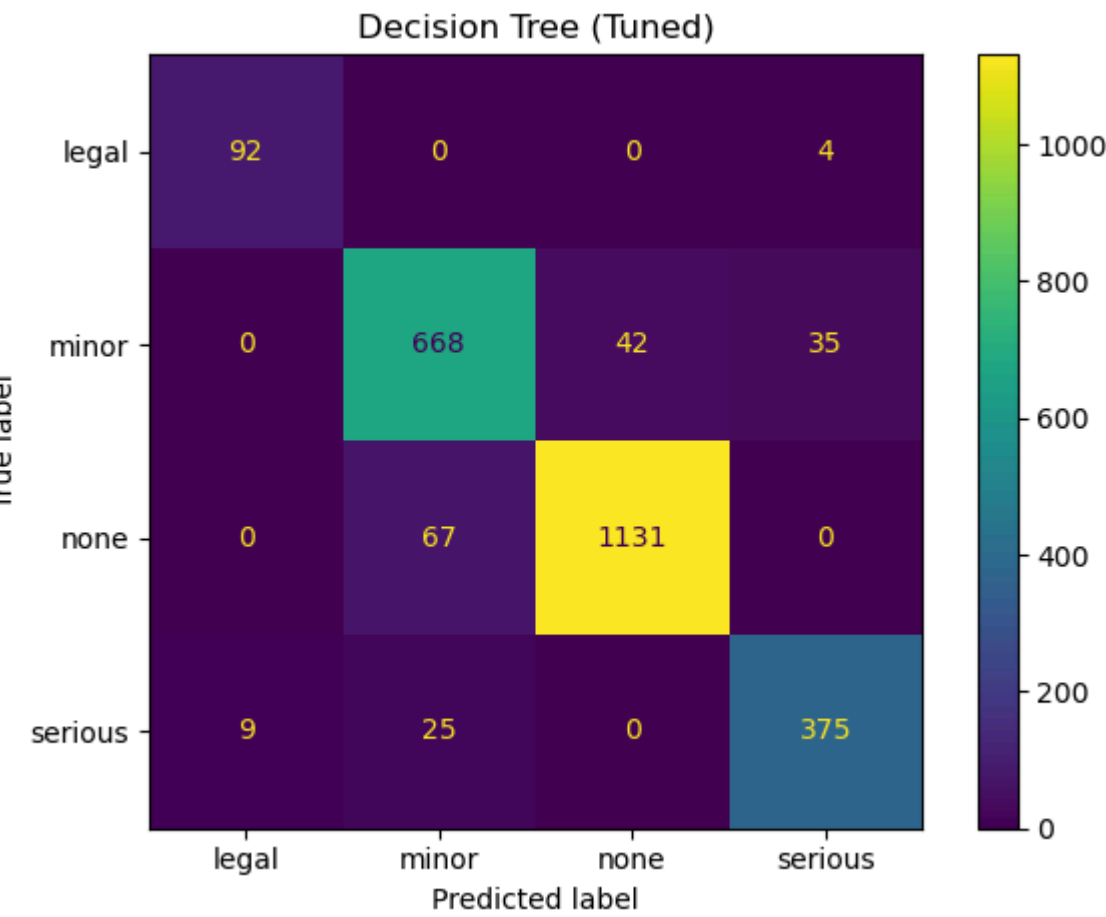


```
plt.xlabel('Max Depth')
plt.ylabel('Accuracy')
plt.title('Decision Tree: Depth vs Accuracy')
plt.legend()
plt.tight_layout()
plt.show()
```

Fitting 5 folds for each of 30 candidates, totalling 150 fits
Best params: {'min_samples_split': 2, 'min_samples_leaf': 1, 'max_depth': 7, 'criterion': 'entropy'}
Best CV macro F1: 0.8969

=== Decision Tree (Tuned) ===

	precision	recall	f1-score	support
legal escalation	0.91	0.96	0.93	96
minor escalation	0.88	0.90	0.89	745
no escalation	0.96	0.94	0.95	1198
serious escalation	0.91	0.92	0.91	409
accuracy			0.93	2448
macro avg	0.91	0.93	0.92	2448
weighted avg	0.93	0.93	0.93	2448



Random Forest

Figures 9 and 10

```
In [49]: from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV

param_dist_rf = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, 30, None],
```

```
'min_samples_split': [2, 5, 10],
'min_samples_leaf': [1, 2, 4],
'max_features': ['sqrt', 'log2']}]

rf = RandomForestClassifier(class_weight='balanced', random_state=42, n_jobs=-1)

rf_search = RandomizedSearchCV( #Help from video https://youtu.be/_QuGM_FW9eo?si=CEfpJch2fv1R9HhP
    rf, param_dist_rf,
    n_iter=20,
    scoring='f1_macro',
    cv=5,
    random_state=42,
    n_jobs=-1,
    verbose=1)

rf_search.fit(X_train_scaled, y_train)

print(f"Best params: {rf_search.best_params_}")
print(f"Best CV macro F1: {rf_search.best_score_:.4f}")

best_rf = rf_search.best_estimator_
y_pred_rf = best_rf.predict(X_test_scaled)

print("\n=== Random Forest (Tuned) ===")
print(classification_report(y_test, y_pred_rf,
    target_names=['legal escalation', 'minor escalation', 'no escalation', 'serious escalation']))

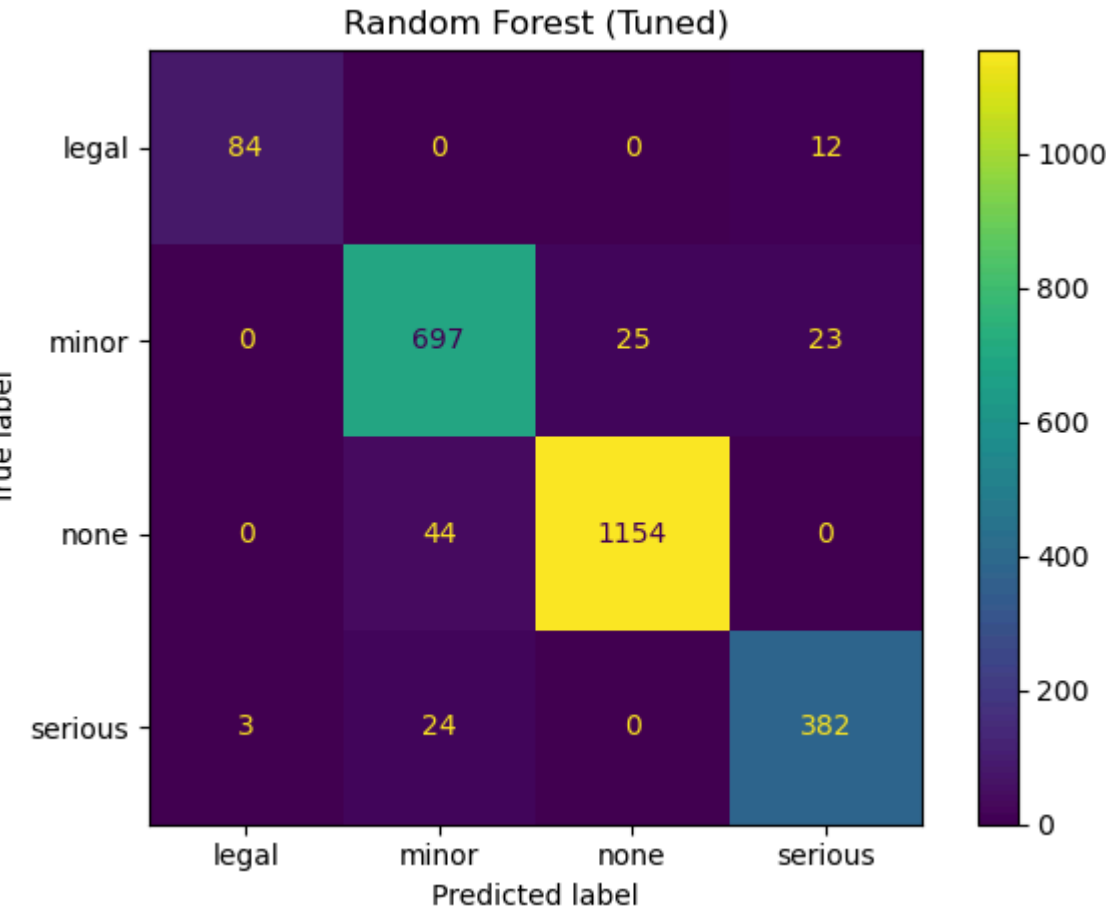
ConfusionMatrixDisplay.from_estimator(best_rf, X_test_scaled, y_test,
    display_labels=['legal', 'minor', 'none', 'serious'])
plt.title("Random Forest (Tuned)")
plt.tight_layout()
plt.show()

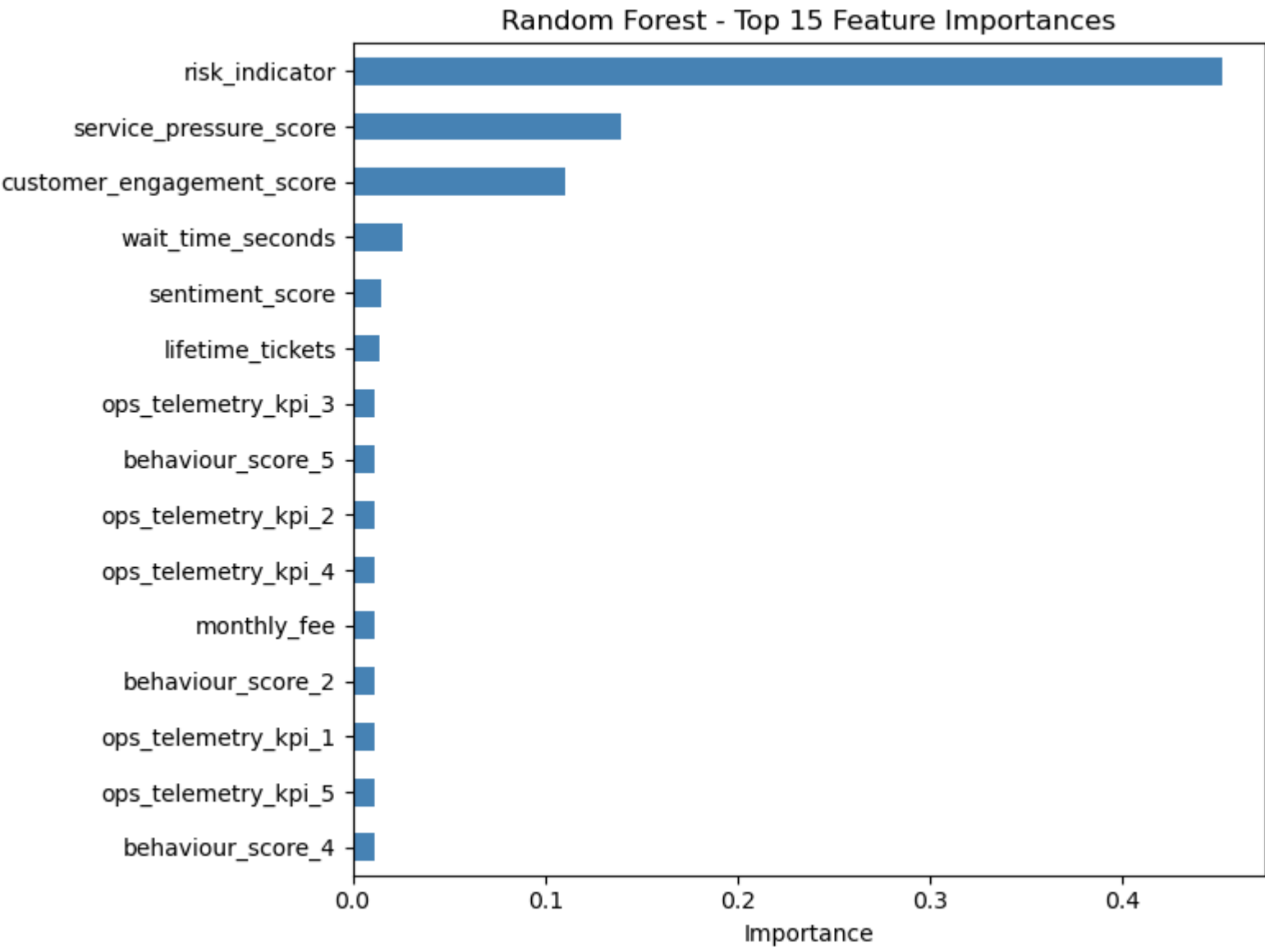
# Feature importance
importances = pd.Series(best_rf.feature_importances_, index=X_train.columns)
top15 = importances.nlargest(15)

plt.figure(figsize=(8, 6))
top15.sort_values().plot(kind='barh', color='steelblue')
plt.title('Random Forest - Top 15 Feature Importances')
plt.xlabel('Importance')
plt.tight_layout()
plt.show()
```

Fitting 5 folds for each of 20 candidates, totalling 100 fits
Best params: {'n_estimators': 200, 'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'sqrt', 'max_depth': 20}
Best CV macro F1: 0.9168

=== Random Forest (Tuned) ===				
	precision	recall	f1-score	support
legal escalation	0.97	0.88	0.92	96
minor escalation	0.91	0.94	0.92	745
no escalation	0.98	0.96	0.97	1198
serious escalation	0.92	0.93	0.92	409
accuracy			0.95	2448
macro avg	0.94	0.93	0.93	2448
weighted avg	0.95	0.95	0.95	2448





Gradient Boosting

Figure 11

```
In [50]: #Help from video https://www.youtube.com/watch?v=aL0QD66Sj0g
# https://youtu.be/E2mCaIZNE2g?si=2v5NA876VXpcY6om
from sklearn.ensemble import HistGradientBoostingClassifier

param_dist_xgb = {
    'max_iter': [100, 200, 300],
    'learning_rate': [0.05, 0.1, 0.2],
    'max_depth': [3, 5, 7],
    'min_samples_leaf': [10, 20, 30],
    'l2_regularization': [0, 1, 10]}

hgb = HistGradientBoostingClassifier(class_weight='balanced', random_state=42)

hgb_search = RandomizedSearchCV(
    hgb, param_dist_xgb,
    n_iter=20,
    scoring='f1_macro',
    cv=5,
    random_state=42,
    n_jobs=-1,
    verbose=1)

hgb_search.fit(X_train_scaled, y_train)

print(f"Best params: {hgb_search.best_params_}")
print(f"Best CV macro F1: {hgb_search.best_score_:.4f}")

best_hgb = hgb_search.best_estimator_
y_pred_hgb = best_hgb.predict(X_test_scaled)

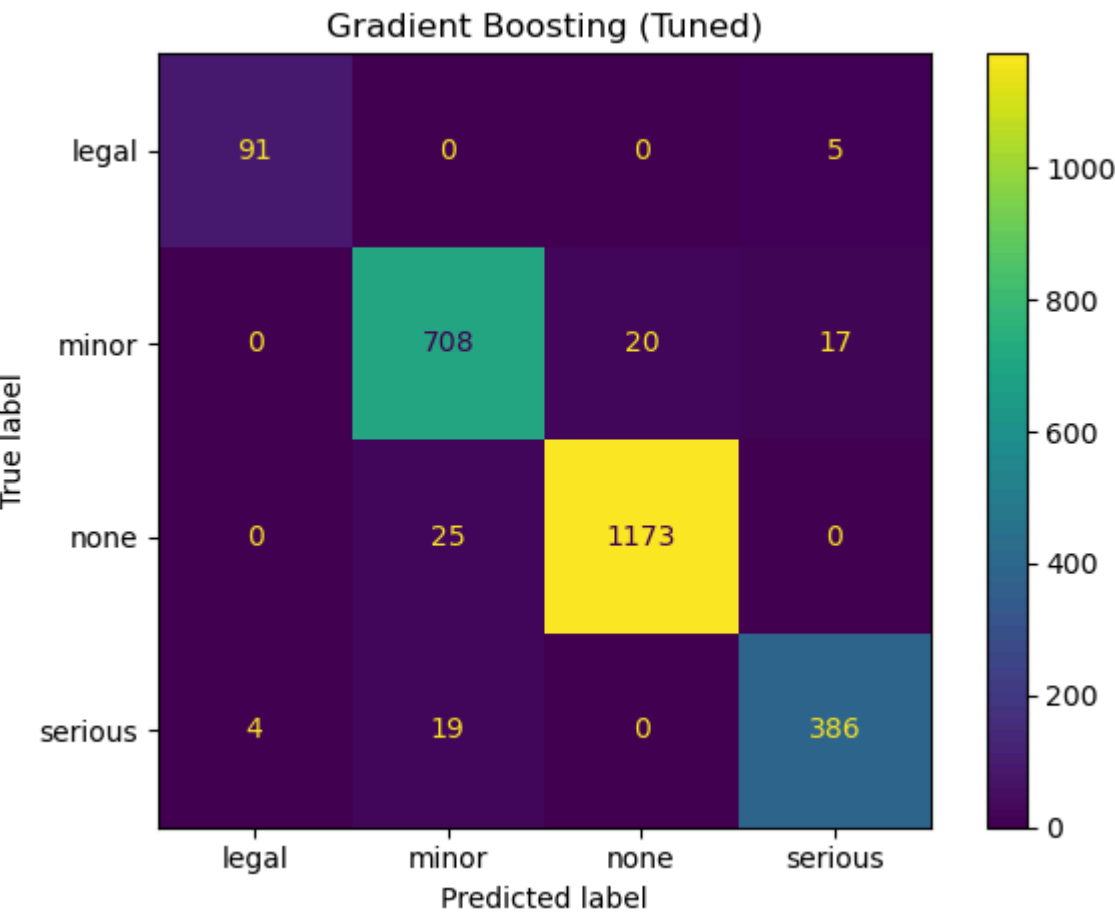
print("\n=== Gradient Boosting (Tuned) ===")
print(classification_report(y_test, y_pred_hgb,
    target_names=['legal escalation', 'minor escalation', 'no escalation', 'serious escalation']))

ConfusionMatrixDisplay.from_estimator(best_hgb, X_test_scaled, y_test,
    display_labels=['legal', 'minor', 'none', 'serious'])
plt.title("Gradient Boosting (Tuned)")
plt.tight_layout()
plt.show()
```


Fitting 5 folds for each of 20 candidates, totalling 100 fits
Best params: {'min_samples_leaf': 30, 'max_iter': 200, 'max_depth': 3, 'learning_rate': 0.1, 'l2_regularization': 1}
Best CV macro F1: 0.9405

=== Gradient Boosting (Tuned) ===

	precision	recall	f1-score	support
legal escalation	0.96	0.95	0.95	96
minor escalation	0.94	0.95	0.95	745
no escalation	0.98	0.98	0.98	1198
serious escalation	0.95	0.94	0.94	409
accuracy			0.96	2448
macro avg	0.96	0.96	0.96	2448
weighted avg	0.96	0.96	0.96	2448



Comparison Visuals

Figures 12 and 13

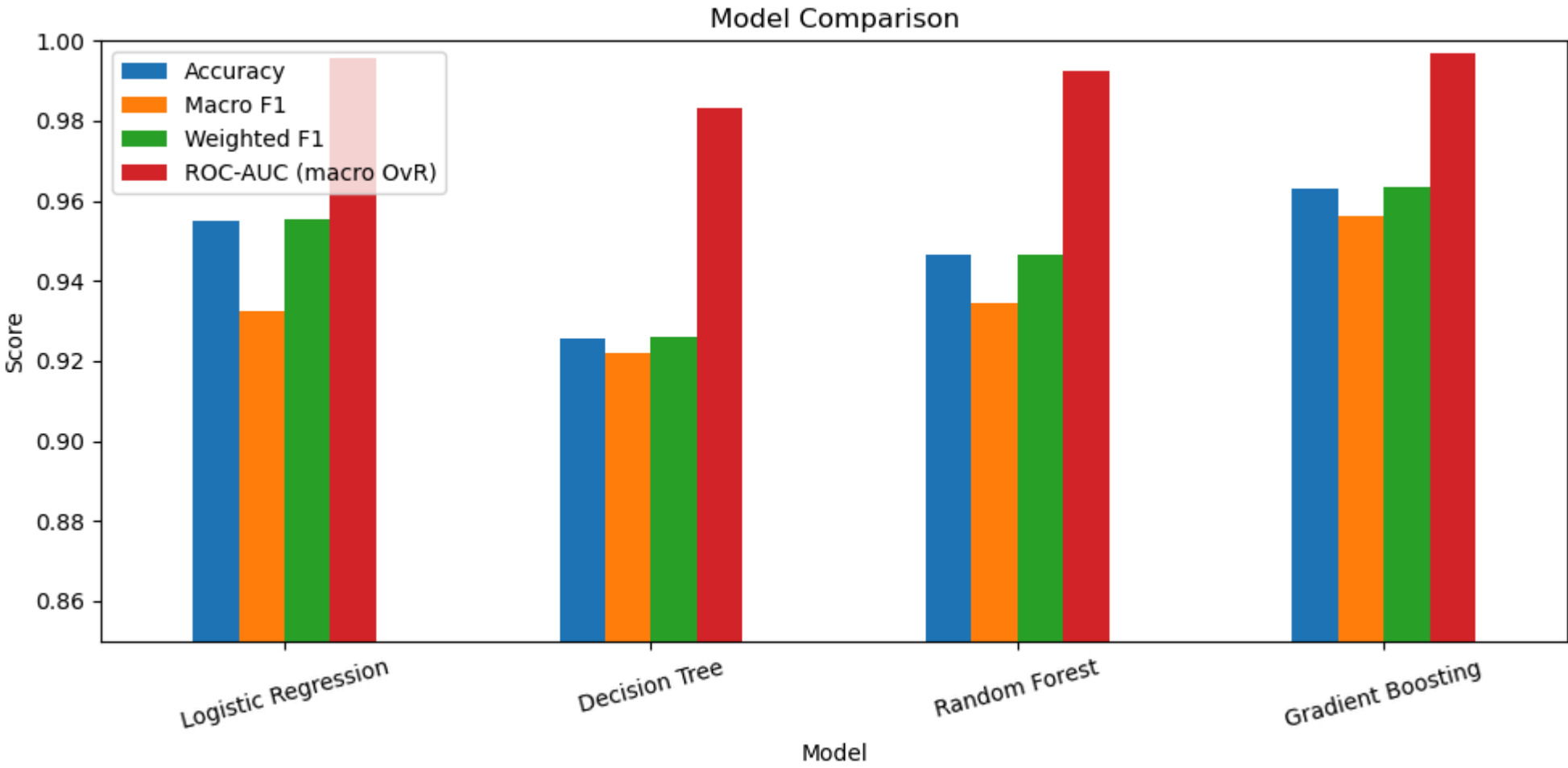
```
In [51]: from sklearn.metrics import roc_auc_score, accuracy_score, f1_score
# Help from video https://youtu.be/4jRBRDbJemM?si=SolTiefbj6wPUVJ
models = {
    'Logistic Regression': (lr_baseline, y_pred_lr),
    'Decision Tree':      (best_dt,    y_pred_dt),
    'Random Forest':      (best_rf,    y_pred_rf),
    'Gradient Boosting':  (best_hgb,   y_pred_hgb)}

results = []
for name, (model, y_pred) in models.items():
    acc      = accuracy_score(y_test, y_pred)
    mac_f1   = f1_score(y_test, y_pred, average='macro')
    wgt_f1   = f1_score(y_test, y_pred, average='weighted')
    roc_auc  = roc_auc_score(y_test,
                             model.predict_proba(X_test_scaled),
                             multi_class='ovr', average='macro')
    results.append({
        'Model': name,
        'Accuracy': round(acc, 4),
        'Macro F1': round(mac_f1, 4),
        'Weighted F1': round(wgt_f1, 4),
        'ROC-AUC (macro OvR)': round(roc_auc, 4)})

results_df = pd.DataFrame(results).set_index('Model')
print(results_df.to_string())

# Visual bar chart comparison
results_df[['Accuracy', 'Macro F1', 'Weighted F1', 'ROC-AUC (macro OvR)']].plot(
    kind='bar', figsize=(10, 5), ylim=(0.85, 1.0), rot=15)
plt.title('Model Comparison')
plt.ylabel('Score')
plt.tight_layout()
plt.show()
```


	Accuracy	Macro F1	Weighted F1	ROC-AUC (macro OvR)
Model				
Logistic Regression	0.9551	0.9326	0.9554	0.9956
Decision Tree	0.9257	0.9218	0.9259	0.9833
Random Forest	0.9465	0.9343	0.9467	0.9925
Gradient Boosting	0.9632	0.9562	0.9633	0.9970



```
In [52]: import shap

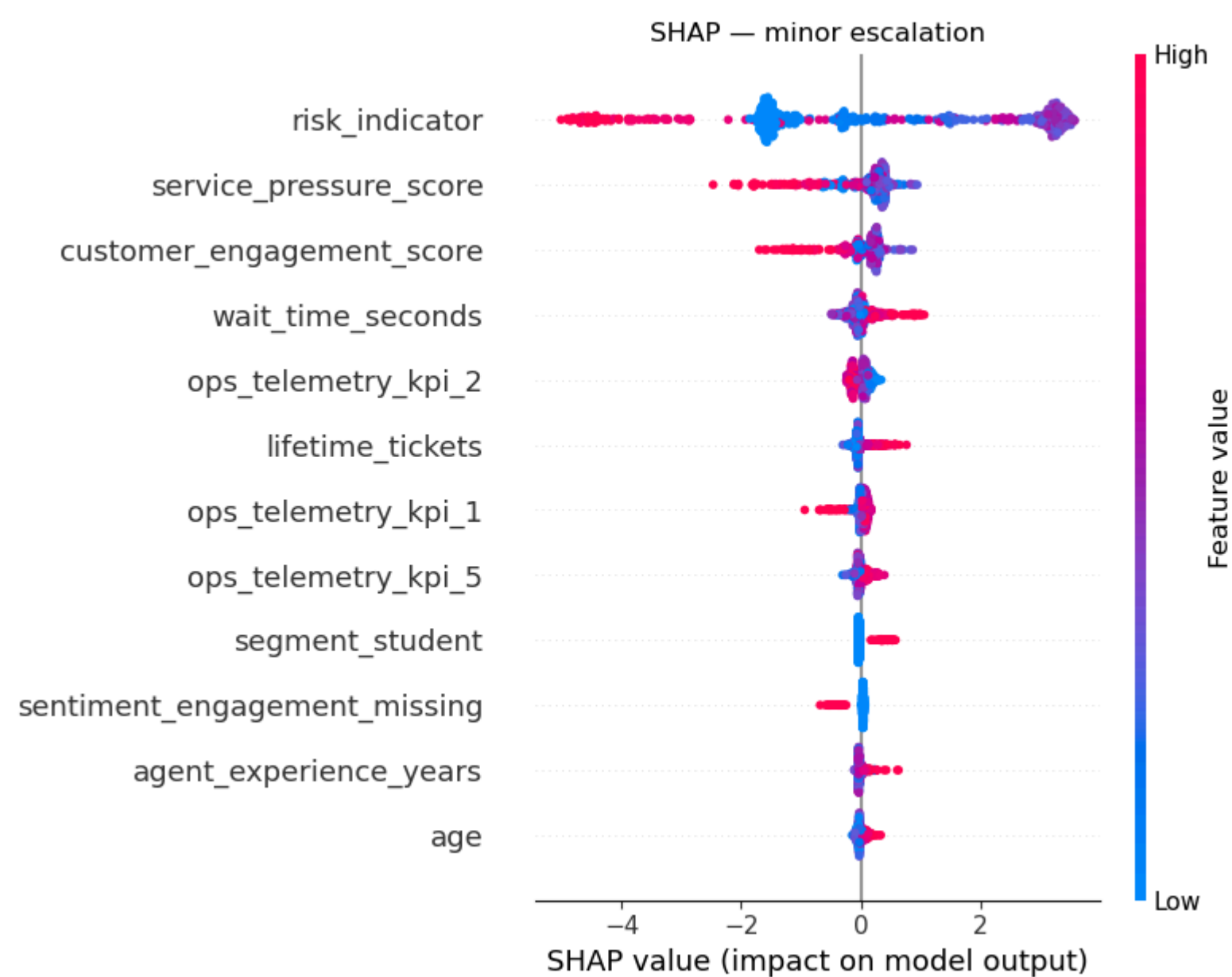
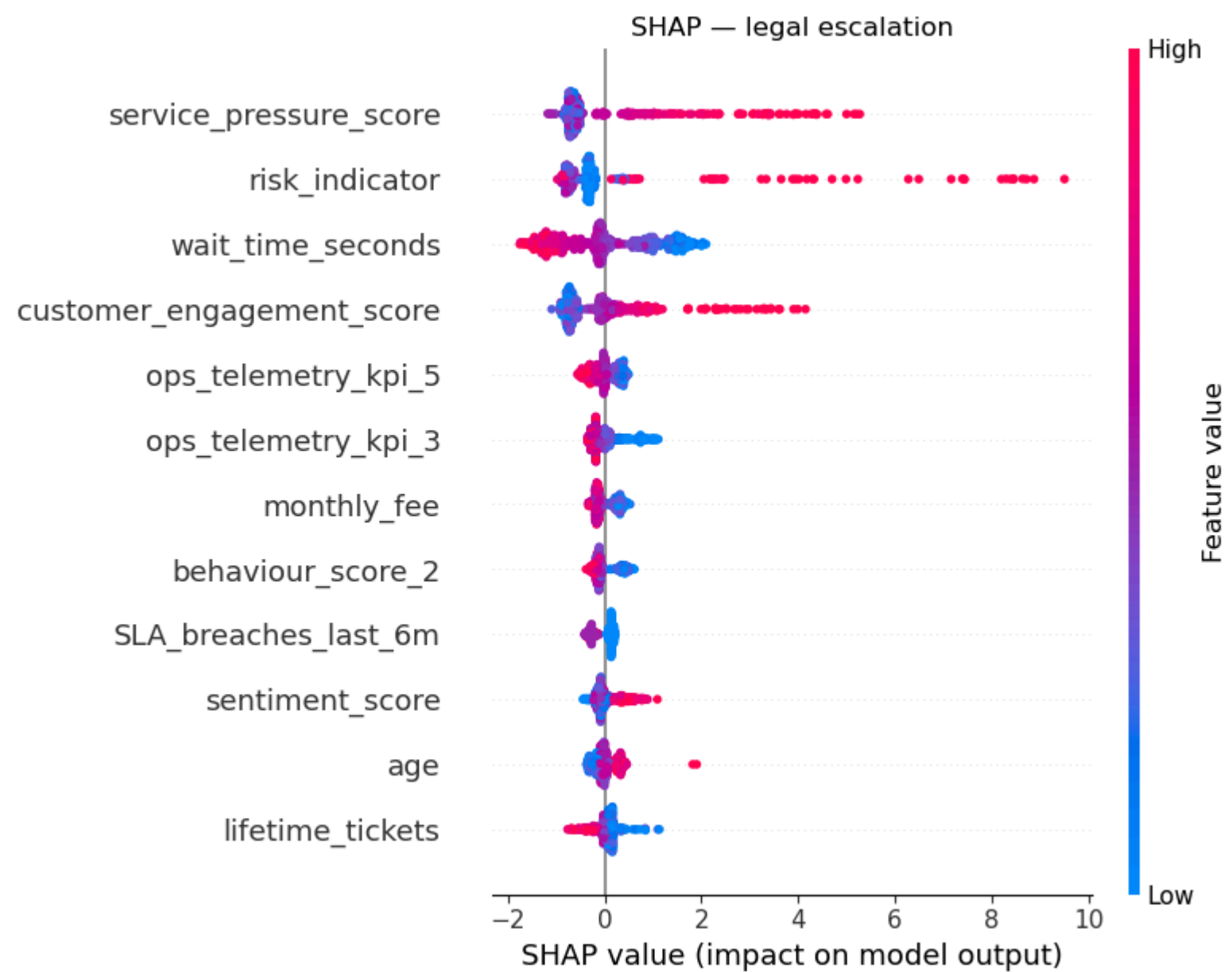
explainer = shap.TreeExplainer(best_hgb)

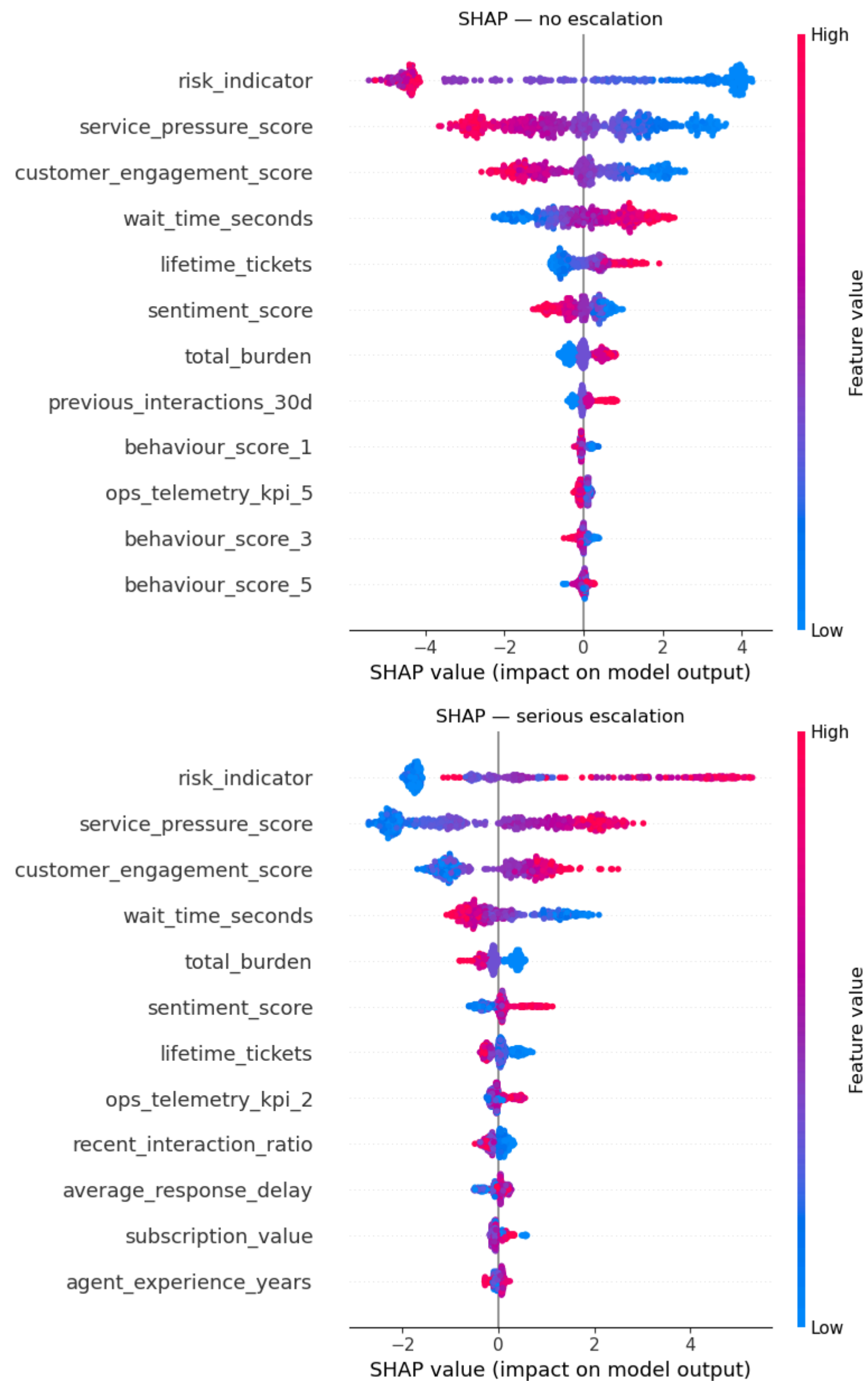
X_test_df = pd.DataFrame(X_test_scaled[:500], columns=X_train.columns)
shap_values = explainer.shap_values(X_test_df)

class_names = ['legal escalation', 'minor escalation', 'no escalation', 'serious escalation']

# Check shape to handle both list and array outputs. Help from video https://www.youtube.com/watch?v=L8_sVRhBDLU
print(f"SHAP values type: {type(shap_values)}")
if isinstance(shap_values, list):
    print(f"Number of classes: {len(shap_values)}, shape per class: {shap_values[0].shape}")
    for i, cls in enumerate(class_names):
        shap.summary_plot(shap_values[i], X_test_df, max_display=12, show=False)
        plt.title(f'SHAP - {cls}')
        plt.tight_layout()
        plt.show()
else:
    print(f"3D array shape: {shap_values.shape}")
    for i, cls in enumerate(class_names):
        shap.summary_plot(shap_values[:, :, i], X_test_df, max_display=12, show=False)
        plt.title(f'SHAP - {cls}')
        plt.tight_layout()
        plt.show()

SHAP values type: <class 'numpy.ndarray'>
3D array shape: (500, 76, 4)
```





Unsupervised Learning Using Clustering Algorithms

- Perform clustering after removing the target variable 'escalation_level'.
- Choose any data representation appropriate (e.g. encoding, normalisation, dimensionality reduction as needed).
- Apply different clustering methods (e.g. k-Means, hierarchical clustering, etc.) and compare them.
- Use suitable visualisation techniques (e.g. scatterplots, PCA, t-SNE) to illustrate cluster structure.
- Analyse how clusters relate to escalation levels.

Data Preparation for Clustering

```
In [53]: from sklearn.decomposition import PCA #Help from video https://youtu.be/QdBy02ExhGI?si=c12rQDKvB0b_36cr
#Removes the target, scales features, uses PCA to reduce to two dimensions
X_clust = df.drop(columns=['escalation_level_enc', 'escalation_level']) # Drop target

# Scale
X_scaled = StandardScaler().fit_transform(X_clust)

# PCA for all plots
pca = PCA(n_components=2, random_state=42)
X_2d = pca.fit_transform(X_scaled)

print(f"Shape ready for clustering: {X_scaled.shape}")
```

Shape ready for clustering: (12240, 76)

Comparing Clustering Models

```
In [54]: from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN #Help from video https://youtu.be/EZ-M2Imi38A?si=JgLS-a0b-XcyDpAg
from sklearn.metrics import silhouette_score, adjusted_rand_score

km = KMeans(n_clusters=4, n_init=10, random_state=42) #Groups customers into four clusters
agg = AgglomerativeClustering(n_clusters=4, linkage='ward') #Merges the most similar customers into clusters
db = DBSCAN(eps=3, min_samples=10) #Looks for dense crowds of points

km_labels = km.fit_predict(X_2d)
agg_labels = agg.fit_predict(X_2d)
db_labels = db.fit_predict(X_2d)

y_labels = df['escalation_level'].values # String labels, not encoded

print("K-Means    silhouette:", round(silhouette_score(X_2d, km_labels), 4),
      " ARI:", round(adjusted_rand_score(y_labels, km_labels), 4))
print("Agglom.    silhouette:", round(silhouette_score(X_2d, agg_labels), 4),
      " ARI:", round(adjusted_rand_score(y_labels, agg_labels), 4))
n_db = len(set(db_labels)) - (1 if -1 in db_labels else 0)
print(f"DBSCAN    clusters found: {n_db}, noise points: {(db_labels==-1).sum()}")
```

K-Means silhouette: 0.3486 ARI: 0.2082
Agglom. silhouette: 0.3 ARI: 0.1818
DBSCAN clusters found: 1, noise points: 0

Visualising Cluster Structure

Figure 14

```
In [55]: from sklearn.manifold import TSNE

# t-SNE
X_tsne = TSNE(n_components=2, perplexity=30, random_state=42).fit_transform(X_2d)

palette = {'no escalation': '#009E73', 'minor escalation': '#56B4E9',
           'serious escalation': '#E69F00', 'legal escalation': '#D55E00'}

fig, axes = plt.subplots(2, 3, figsize=(16, 9))

# PCA
for lvl, col in palette.items():
    m = y_labels == lvl
    axes[0,0].scatter(X_2d[m,0], X_2d[m,1], c=col, s=5, alpha=0.4, label=lvl)
axes[0,0].set_title('True Labels (PCA)')
axes[0,0].legend(fontsize=7, markerscale=3)

axes[0,1].scatter(X_2d[:,0], X_2d[:,1], c=km_labels, cmap='tab10', s=5, alpha=0.4)
axes[0,1].set_title('K-Means (PCA)')

axes[0,2].scatter(X_2d[:,0], X_2d[:,1], c=agg_labels, cmap='tab10', s=5, alpha=0.4)
axes[0,2].set_title('Agglomerative (PCA)')

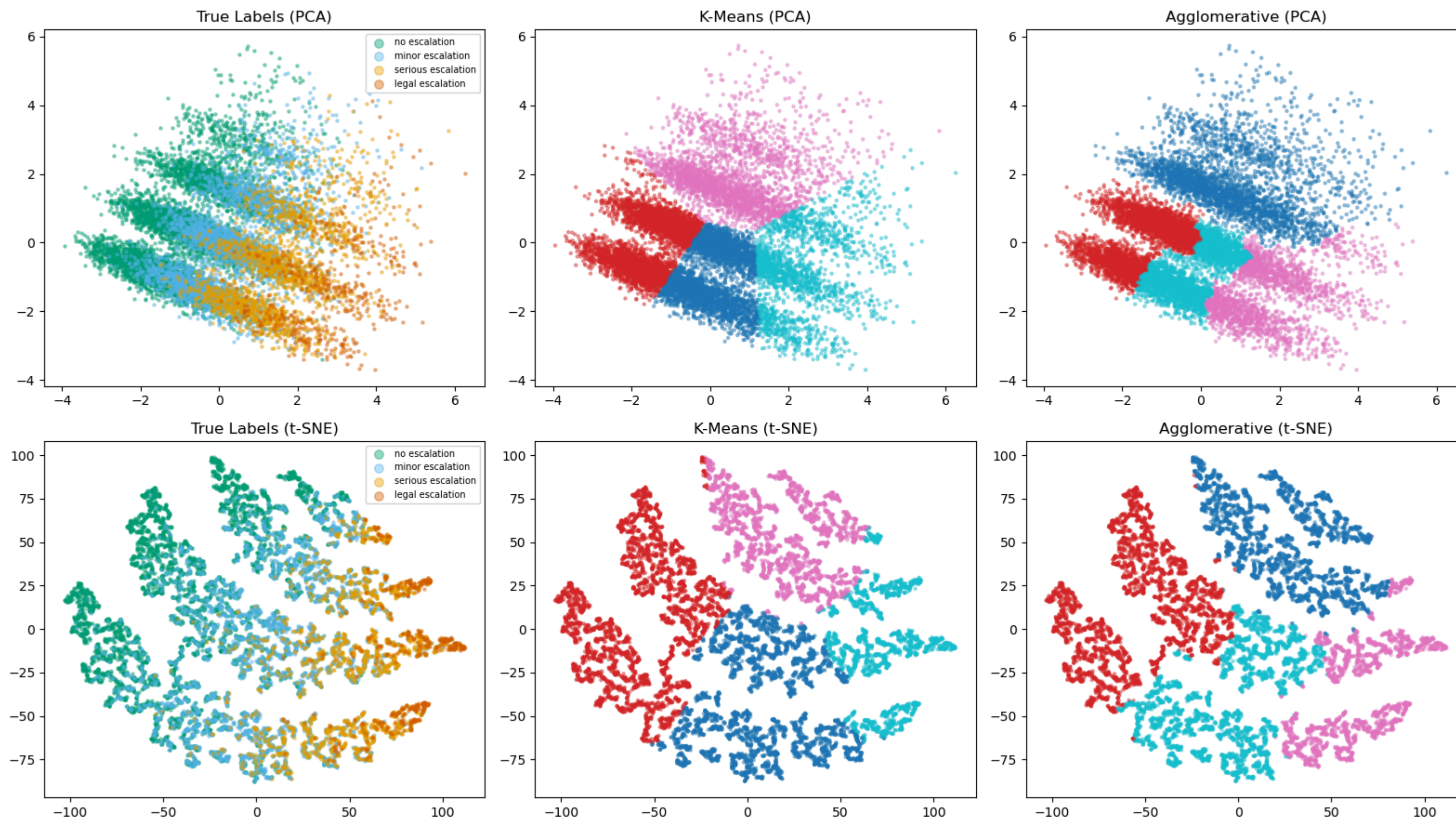
# t-SNE
for lvl, col in palette.items():
    m = y_labels == lvl
    axes[1,0].scatter(X_tsne[m,0], X_tsne[m,1], c=col, s=5, alpha=0.4, label=lvl)
axes[1,0].set_title('True Labels (t-SNE)')
axes[1,0].legend(fontsize=7, markerscale=3)

axes[1,1].scatter(X_tsne[:,0], X_tsne[:,1], c=km_labels, cmap='tab10', s=5, alpha=0.4)
axes[1,1].set_title('K-Means (t-SNE)')

axes[1,2].scatter(X_tsne[:,0], X_tsne[:,1], c=agg_labels, cmap='tab10', s=5, alpha=0.4)
axes[1,2].set_title('Agglomerative (t-SNE)')
```



```
plt.tight_layout()
plt.show()
```



Visualising Cluster to Escalation Level

Figure 15

```
In [56]: label_order = ['no escalation', 'minor escalation', 'serious escalation', 'legal escalation']

res = pd.DataFrame({'escalation': y_labels, 'kmeans': km_labels, 'agg': agg_labels})

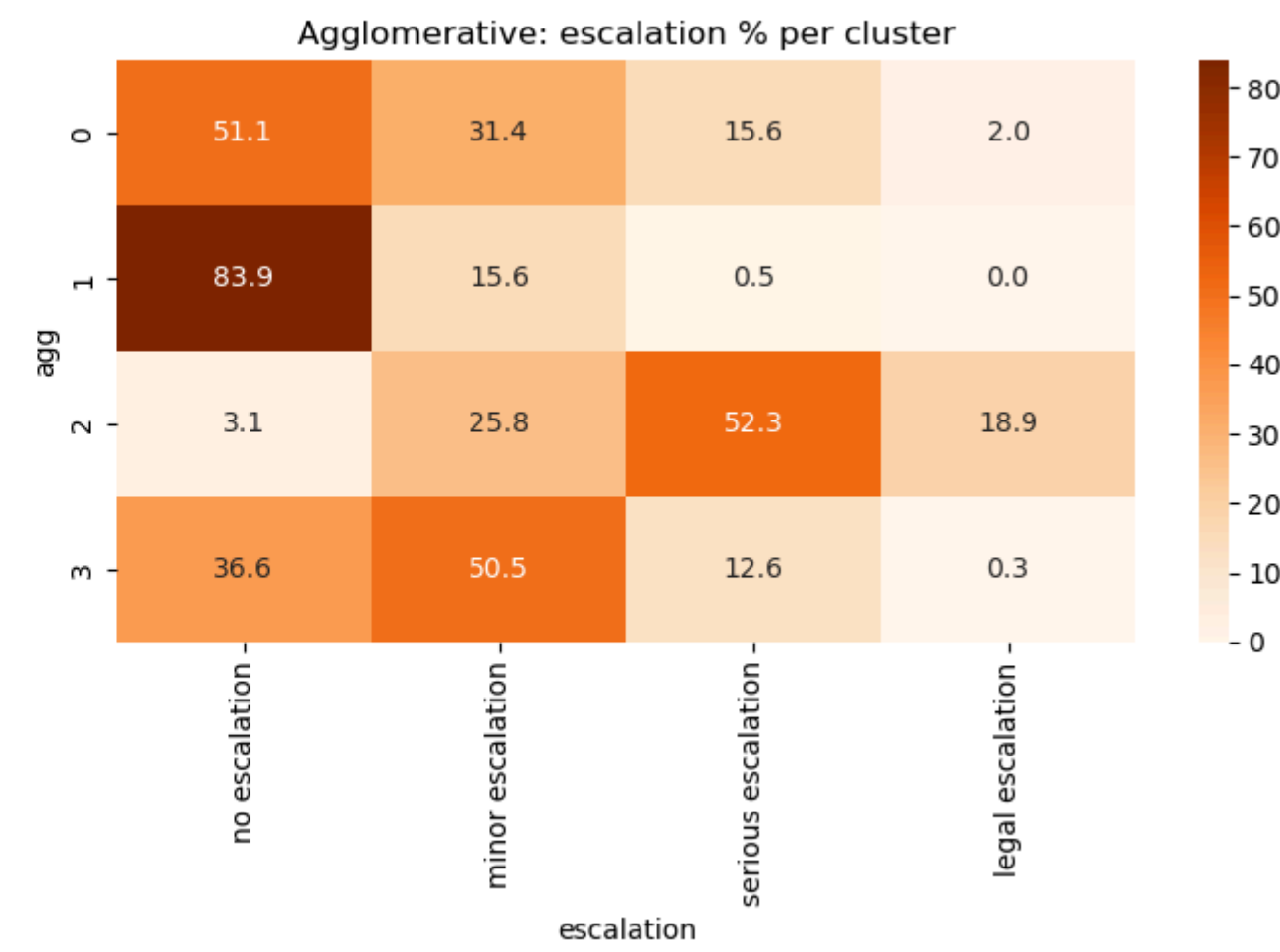
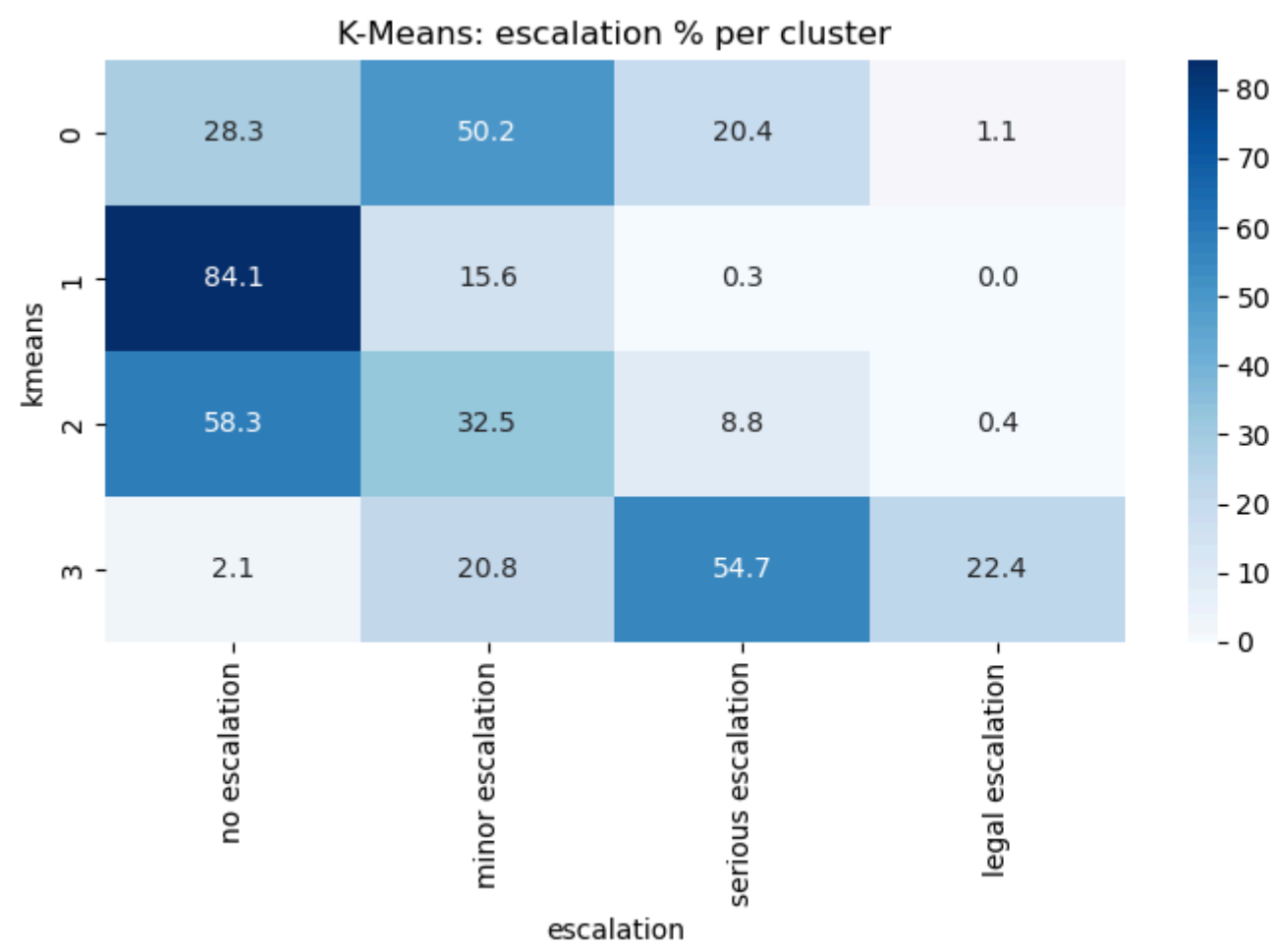
ct_km = pd.crosstab(res['kmeans'], res['escalation'], normalize='index')[label_order] * 100 #How many customers from K Means cluster belong to each escalation level
ct_agg = pd.crosstab(res['agg'], res['escalation'], normalize='index')[label_order] * 100 #For Agglomerative clustering

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.heatmap(ct_km, annot=True, fmt='.1f', cmap='Blues', ax=axes[0])
axes[0].set_title('K-Means: escalation % per cluster')

sns.heatmap(ct_agg, annot=True, fmt='.1f', cmap='Oranges', ax=axes[1])
axes[1].set_title('Agglomerative: escalation % per cluster')

plt.tight_layout()
plt.show()
```



Variable	Section	Type	Description	Values / Notes
escalation_level	Original	Categorical	Target variable. Outcome of the customer interaction	No escalation, Minor, Serious, Legal
age	Original	Numeric	Customer age in years	18-90 (after cleaning)
tenure_months	Original	Numeric	Months the customer has been with the company	0-141
loyalty_status	Original	Categorical	Customer loyalty tier	Bronze, Silver, Gold, Platinum
segment	Original	Categorical	Customer account type	Retail, Business, Enterprise, Student, Other
region	Original	Categorical	UK region of the customer	London, South East, Midlands, North West, Scotland, Wales
industry	Original	Categorical	Industry sector of the customer	Retail, Finance, Hospitality, Tech, Education, Healthcare
channel	Original	Categorical	How the customer contacted support	Chat, Phone, Email, In-app
issue_category	Original	Categorical	Broad category of the issue raised	Technical, Billing, Returns, Orders, Account, Other
issue_subtype	Original	Categorical	Specific type of issue (cleaned of typos)	11 subtypes e.g. Wrong item, Payment failed
product_line	Original	Categorical	Which product the customer is subscribed to	Basic, Standard, Premium, Enterprise
device_type	Original	Categorical	Device used to contact support	Mobile, Desktop, Web, Call-centre
time_of_day_bucket	Original	Categorical	Time of day the interaction occurred	Morning, Afternoon, Evening, Night
wait_time_seconds	Original	Numeric	Seconds customer waited before being connected	0-454 (after capping)
previous_interactions_30d	Original	Numeric	Number of interactions in the last 30 days	0-5 (after capping)
resolution_time_minutes	Original	Numeric	Time taken to resolve the issue in minutes	1-111 (after capping)
agent_experience_years	Original	Numeric	Years of experience of the handling agent	0-12; 1.8% missing
first_contact_resolution	Original	Binary	Whether issue was resolved on first contact	1 = Yes, 0 = No
number_of_transfers	Original	Numeric	Times customer was transferred between agents	0-2.5 (after capping)
SLA_breaches_last_6m	Original	Numeric	Number of SLA breaches in the last 6 months	0-2.5 (after capping)
lifetime_tickets	Original	Numeric	Total support tickets ever raised by customer	1-43 (after capping)
average_response_delay	Original	Numeric	Average delay in agent responses	0-5.67 (after capping)
sentiment_score	Original	Numeric	Sentiment score from interaction text	-2.68 to 2.67; 5.1% missing
monthly_fee	Original	Numeric	Monthly subscription fee paid by the customer	5-88.78 (after capping)
subscription_value_str	Original	String	Annual subscription value - original string format	Converted to numeric; 3.23% missing
churn_risk_band	Original	Categorical	Assessed churn risk category	Low, Medium, High
priority_flag	Original	Binary	Whether the customer is flagged as priority	Yes, No
behaviour_score_1	Original	Numeric	Behavioural metric 1	10.7-90.95
behaviour_score_2	Original	Numeric	Behavioural metric 2	9.5-90.3
behaviour_score_3	Original	Numeric	Behavioural metric 3	9.9-90.7
behaviour_score_4	Original	Numeric	Behavioural metric 4	9.6-90.4
behaviour_score_5	Original	Numeric	Behavioural metric 5	8.95-90.95
ops_telemetry_kpi_1	Original	Numeric	Operational telemetry KPI 1 (z-scored)	-3.75 to 4.09
ops_telemetry_kpi_2	Original	Numeric	Operational telemetry KPI 2 (z-scored)	-3.72 to 3.39
ops_telemetry_kpi_3	Original	Numeric	Operational telemetry KPI 3 (z-scored)	-3.59 to 3.79
ops_telemetry_kpi_4	Original	Numeric	Operational telemetry KPI 4 (z-scored)	-3.64 to 3.60
ops_telemetry_kpi_5	Original	Numeric	Operational telemetry KPI 5 (z-scored)	-3.76 to 3.73
post_interaction_quality_score	Original	Numeric	Quality score recorded after the interaction	-2.21 to 10.88
risk_indicator	Original	Numeric	Composite risk score for the customer	0-61.20; 1.46% missing
service_pressure_score	Original	Numeric	Pressure on the service team at time of contact	0-49.25; 1.51% missing
customer_engagement_score	Original	Numeric	How engaged the customer is	0-29.88; 6.68% missing
subscription_value	Engineered	Numeric	Cleaned numeric version of subscription_value_str	50-1,080 (after capping)
sentiment_engagement_missing	Engineered	Binary	Flag: 1 if sentiment_score or customer_engagement_score was missing	0 = complete, 1 = at least one missing
total_burden	Engineered	Numeric	Sum of SLA_breaches + number_of_transfers + previous_interactions_30d	Higher = more friction experienced
high_risk	Engineered	Binary	1 if risk_indicator is above the 75th percentile	0 = standard, 1 = high risk
recent_interaction_ratio	Engineered	Numeric	Ratio of previous_interactions_30d to lifetime_tickets	Higher = recent contact spike relative to history
escalation_level_enc	Engineered	Numeric	Label-encoded target used in supervised models only	Legal=0, Minor=1, No escalation=2, Serious=3. Dropped before clustering
channel_email / _in-app / _phone	Encoded	Binary	Contact channel (reference: chat)	0 or 1
issue_category_billing / _orders / _other / _returns / _technical	Encoded	Binary	Issue category (reference: account)	0 or 1
issue_subtype_(10 columns)	Encoded	Binary	Issue subtype (reference: damaged item)	0 or 1
product_line_enterprise / _premium / _standard	Encoded	Binary	Product line (reference: basic)	0 or 1
device_type_desktop / _mobile / _web	Encoded	Binary	Device type (reference: call-centre)	0 or 1
region_midlands / _north west / _scotland / _south east / _wales	Encoded	Binary	UK region (reference: london)	0 or 1
industry_finance / _healthcare / _hospitality / _retail / _tech	Encoded	Binary	Industry sector (reference: education)	0 or 1
segment_enterprise / _other / _retail / _student	Encoded	Binary	Customer segment (reference: business)	0 or 1
time_of_day_bucket_evening / _morning / _night	Encoded	Binary	Time of day (reference: afternoon)	0 or 1
churn_risk_band_low / _medium	Encoded	Binary	Churn risk band (reference: high)	0 or 1
loyalty_status_gold / _platinum / _silver	Encoded	Binary	Loyalty tier (reference: bronze)	0 or 1

Travel Log

[2026-04-26 13:02]

Task completed: Performed initial data exploration and familiarization such as key features and basic statistics, identify columns and rows, and inspect unique and missing values

Why I did it: To become familiar with dataset and tasks to be done to prepare for next stage

Tools/commands used (optional): Python

Notes: none

[2026-04-26 16:52]

Task completed: Performed further exploration tasks including plotting distributions for every numeric column using histograms for categoricals and flagging anything skewed. Initial data dictionary created.

Why I did it: Familiarisation/prepare for transformation

Tools/commands used (optional): Python, Seaborn

Notes: Adding final data dictionary at the end

[2026-04-26 18:08]

Task completed: Exploratory visualizations including escalation distribution, compared with numeric features, missingness correlation across columns, sentiment and engagement vs escalation level, and behavior scores distribution

Why I did it: Further exploratory activities

Tools/commands used (optional): Python, Missingno, Seaborn, Matplotlib

Notes: loyalty_status and age - correlation of 0.1. Weak and not meaningful.

[2026-04-27 01:31]

Task completed: Worked on cleaning data by converting dtypes and applying fuzzy matching function to region column

Why I did it: Beginning on cleaning data types and strings to support data transformation and analysis

Tools/commands used (optional): Python, difflib

Notes: Work on other columns with inconsistent spelling

[2026-04-27 17:11]

Task completed: Cleaned strings in issue_subtype column

Why I did it: Needed to prepare for analysis

Tools/commands used (optional): Python, rapidfuzz

Notes: None

[2026-04-28 13:01]

Task completed: Verify no remaining nulls in cleaned strings values, check missingness btw sentiment and customer engagement and create flag before imputing with median, identify outliers for numeric columns

Why I did it: Preprocessing

Tools/commands used (optional): Python

Notes: None

[2026-04-28 19:56]

Task completed: Capped extreme values for numeric columns, address age outliers with imputation, dropped columns that aren't relevant to modeling, addressed class balance and feature engineering, had to fix fuzzy match because there were some misspellings, encoded categoricals

Why I did it: Preprocessing

Tools/commands used (optional): Python, thefuzz process installed(replaced previous libraries)

Notes: String cleaning fix

[2026-04-29 00:36]

Task completed: Needed to fix issues such as addressing outliers before imputing missing values

Why I did it: To prevent incorrect data to be used

Tools/commands used (optional): Python

Notes: Had to restructure order

[2026-05-01 01:32]

Task completed: Work on encoding and start/finish train/test data

Why I did it: Prepare for modeling

Tools/commands used (optional): Python, sklearn (train/test/split and scaler

Notes: None

[2026-05-02 00:10]

Task completed: Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting

Why I did it: Models to predict which level a new interaction will be based on features

Tools/commands used (optional): Python, sklearn

Notes: None

[2026-05-02 22:57]

Task completed: Cluster algorithms and worked on graphs to make sure it is clear

Why I did it: Discover natural customer groupings without using escalation labels and then see whether those clusters align with escalation behavior

Tools/commands used (optional): Python, Pandas, sklearn libraries

Notes: None